
Samsung PIM Simulator Summary

***Intelligent System
Laboratory***

Samsung PIM Simulator CMDs

❑ PIM Instruction Set Architecture

- Simulator의 PIM CMDs

Type	Command	Description	Result (DST)	Operand (SRC0)	Operand (SRC1)
Arithmetic	ADD	Addition	GRF	GRF, BANK, SRF	GRF, BANK, SRF
Arithmetic	MUL	Multiplication	GRF	GRF, BANK	GRF, BANK, SRF
Arithmetic	MAC	Multiply-Accumulate	GRF_B	GRF, BANK	GRF, BANK, SRF
Arithmetic	MAD	Multiply-Add	GRF	GRF, BANK	GRF, BANK, SRF
Data	MOV	load/store data from register to bank	GRF	GRF, SRF	GRF, BANK
Data	FILL	copy data from bank to register	GRF	GRF, BANK	GRF, BANK
Control	NOP	do nothing	-	-	-
Control	JUMP	jump instruction	-	-	-
Control	EXIT	exit instruction	-	-	-

Samsung PIM Simulator CMDs

❑ PIM Instruction Set Architecture

- RISC-style 32비트 Instructions를 Support
- 3가지 Instruction 유형:
 - 4개의 Arithmetic: ADD, MUL, MAC, MAD
 - 2개의 Data Transfer: MOV, FILL
 - 3개의 Control: NOP, JUMP, EXIT
- JUMP Instruction: 0-cycle static branch
 - 미리 programming된 반복 횟수만 지원
- Operand 유형:
 - Vector Register(GRF_A, GRF_B)
 - Scalar Register(SRF)
 - Bank Row Buffer

Samsung PIM Simulator CMDs

❑ PIM Instruction Set Architecture

- PIM Instructions는 CRF에 저장됨, Memory Instructions가 CRF를 trigger하여 target instruction를 수행
 - 각 Memory Instruction이 CRF PC를 increment
- DRAM Instruction이 PIM 연산을 위해 DRAM에서 data를 가져오는 위치를 결정

Samsung PIM Simulator CMDs

❑ Movement of Data

- SB mode: standard DRAM operation
- HAB mode: Allowing concurrent accesses to multiple banks with a single DRAM command
- PIM mode: Triggers the execution of PIM instructions on the CRF by DRAM Command

Mode	Transaction	PIM Instruction	Operation
SB	Read	-	Normal Memory Read
SB	Write	-	Normal Memory Write
HAB	Write	-	PIM Write (Host to PIM Register)
PIM	-	MOV	read or write from bank to PIM Register

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – TEST_F()

- Samsung PIM Simulator는 GoogleTest(gtest)기반의 simulation test를 수행함
- 따라서, flow를 Samsung PIM Simulator가 hbm read operation을 어떻게 test하는지 따라가며 알아본다

./sim --gtest_filter=MemBandwidthFixture.hbm read bandwidth

```
1. #include <bitset>
2. #include <cstdint>
3. #include <iostream>
4. #include <limits>
5. #include <memory>
6. #include <random>
7.
8. #include "Burst.h"
9. #include "MultiChannelMemorySystem.h"
10. #include "gtest/gtest.h"
11. #include "tests/PIMKernel.h"
12. #include "tests/TestCases.h"
13.
14. using namespace std;
15.
16. int main(int argc, char* argv[])
17. {
18.     if (argc > 1)
19.     {
20.         ::testing::InitGoogleTest(&argc, argv);
21.     }
22.     else
23.     {
24.         ::testing::InitGoogleTest(&argc, argv);
25.         testing::GTEST_FLAG(filter) = "-*bw*.*";
26.     }
27.
28.     return RUN_ALL_TESTS();
29. }
```

MainTest.cpp

```
1. TEST_F(MemBandwidthFixture, hbm_read_bandwidth)
2. {
3.     setDataSize(128 * 1024 * 64); // in bytes
4.     uint64_t bw = measureCycle(false);
5.     float effective_bw_ratio = 0.8;
6.     EXPECT_TRUE(bw > 256 * effective_bw_ratio);
7. }
```

MemTestCases.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – SetUp()

- ./sim --gtest_filter=MemBandwidthFixture.hbm_read_bandwidth를 실행 시,
- TEST_F(MemBandwidthFixture, hbm_read_bandwidth)가 실행됨
- 이 때, TEST_F()는 MemBandwidthFixture의 SetUp()을 실행하여 mem에 대한 정보를 initialize함

MemTestCases.cpp

```
1. TEST_F(MemBandwidthFixture, hbm_read_bandwidth)
2. {
3.     setDataSize(128 * 1024 * 64); // in bytes
4.     uint64_t bw = measureCycle(false);
5.     float effective_bw_ratio = 0.8;
6.     EXPECT_TRUE(bw > 256 * effective_bw_ratio);
7. }
```

TestCases.h

```
1. virtual void SetUp()
2. {
3.     cur cycle = 0;
4.     mem = make_shared<MultiChannelMemorySystem>("ini/HBM2_samsung_2M_16B_x64.ini",
5.         "system hbm.ini", ".", "example_app", 256 * 16);
6.     mem_size = (uint64_t)getConfigParam(UINT, "NUM_CHANS") * getConfigParam(UINT, "NUM_RANKS") *
7.         getConfigParam(UINT, "NUM_BANK_GROUPS") * getConfigParam(UINT, "NUM_BANKS") *
8.         getConfigParam(UINT, "NUM_ROWS") * getConfigParam(UINT, "NUM_COLS");
9.     basic_stride =
10.         (getConfigParam(UINT, "JEDEC_DATA_BUS_BITS") * getConfigParam(UINT, "BL") / 8);
11. }
```

```
1. MultiChannelMemorySystem::MultiChannelMemorySystem(const string& deviceIniFilename_,
2.     const string& systemIniFilename_,
3.     const string& pwd_, const string& traceFilename_,
4.     unsigned megsOfMemory_, string* visFilename_)
5. : megsOfMemory(megsOfMemory_),
6.   deviceIniFilename(deviceIniFilename_),
7.   systemIniFilename(systemIniFilename_),
8.   traceFilename(traceFilename_),
9.   pwd(pwd_),
10.  visFilename(visFilename_),
11.  clockDomainCrossover(new ClockDomain::Callback<MultiChannelMemorySystem, void>{
12.      this, &MultiChannelMemorySystem::actual_update}),
13.  csvOut(new CSVWriter(visDataOut))
14. {
15.     currentClockCycle = 0;
16.     if (visFilename)
17.         printf("CC VISFILENAME=%s\n", visFilename->c_str());
18.
19.     if (!isPowerOfTwo(megsOfMemory))
20.     {
21.         ERROR("Please specify a power of 2 memory size");
22.         abort();
23.     }
24.
25.     ConfigurationDB& configDB = ConfigurationDB::getDB();
26.     configDB.initialize();
27.     configDB.updateFromFile(deviceIniFilename);
28.     configDB.updateFromFile(systemIniFilename);
29.
30.     addrMapping = new AddrMapping();
31.     configuration = new Configuration(*addrMapping);
32.     numFence = new unsigned[configuration->NUM_CHANS]();
33.
34.     for (size_t i = 0; i < configuration->NUM_CHANS; i++)
35.     {
36.         MemorySystem* channel = new MemorySystem(i, megsOfMemory / configuration->NUM_CHANS,
37.             (*csvOut), dramsimLog, *configuration);
38.         channels.push_back(channel);
39.     }
40. }
```

callback 등록

Clock 초기화

각 .ini의 내용을
configDB에 저장

각 .ini의 내용을 기반해
address bit 정보 가져옴

Channel 개수만큼
MemorySystem 생성

MultiChannelMemorySystem.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – SetUp()

- Memory 생성 시
 - Channel개수만큼 MemorySystem 생성 → 각 MemorySystem 마다 Rank개수만큼 Rank Class 생성

```
1. MultiChannelMemorySystem::MultiChannelMemorySystem(const string& deviceIniFilename_,
2.                                                       const string& systemIniFilename_,
3.                                                       const string& pwd_, const string& traceFilename_,
4.                                                       unsigned megsOfMemory_, string* visFilename_)
5. : megsOfMemory(megsOfMemory_),
6.   deviceIniFilename(deviceIniFilename_),
7.   systemIniFilename(systemIniFilename_),
8.   traceFilename(traceFilename_),
9.   pwd(pwd_),
10.  visFilename(visFilename_),
11.  clockDomainCrossover(new ClockDomain::Callback<MultiChannelMemorySystem, void>(
12.      this, &MultiChannelMemorySystem::actual_update)),
13.  csvOut(new CSVWriter(visDataOut))
14. {
15.     currentClockCycle = 0;
16.     if (visFilename)
17.         printf("CC VISFILENAME=%s\n", visFilename->c_str());
18.     if (!isPowerOfTwo(megsOfMemory))
19.     {
20.         ERROR("Please specify a power of 2 memory size");
21.         abort();
22.     }
23. }
24. ConfigurationDB& configDB = ConfigurationDB::getDB();
25. configDB.initialize();
26. configDB.updateFromFile(deviceIniFilename);
27. configDB.updateFromFile(systemIniFilename);
28.
29. addrMapping = new AddrMapping();
30. configuration = new Configuration(*addrMapping);
31. numFence = new unsigned[configuration->NUM_CHANS]();
32.
33. for (size_t i = 0; i < configuration->NUM_CHANS; i++)
34. {
35.     MemorySystem* channel = new MemorySystem(i, megsOfMemory / configuration->NUM_CHANS,
36.                                               ("csvOut", dramsimLog, "configuration");
37.     channels.push_back(channel);
38. }
39.
40. }
```

Channel 개수만큼
MemorySystem 생성

```
1. MemorySystem::MemorySystem(unsigned id, unsigned int megsOfMemory, CSVWriter& csvOut_,
2.                               ostream& simLog, Configuration& configuration)
3. : dramsimLog(simLog),
4.   ReturnReadData(NULL),
5.   WriteDataDone(NULL),
6.   systemID(id),
7.   csvOut(csvOut_),
8.   numOnTheFlyTransactions(0),
9.   config(configuration)
10. {
11.     currentClockCycle = 0;
12.
13.     DEBUG("===== MemorySystem " << systemID << " =====");
14.     uint64_t bytePerRank = static_cast<uint64_t>{
15.         (getConfigParam(UINT, "NUM_ROWS") * getConfigParam(UINT, "DEVICE_WIDTH")) *
16.         (getConfigParam(UINT, "NUM_COLS") * getConfigParam(UINT, "NUM_BANKS")) *
17.         (getConfigParam(UINT64, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH")) /
18.         8);
19.     uint64_t megsOfStoragePerRank = bytePerRank >> 20; // bytes to MB (2^20 = 1,048,576 = Mega)
20.
21.     num_ranks_ = (megsOfMemory / Byte2MB(bytePerRank));
22.
23.     // If this is set, effectively override the number of ranks
24.     if (megsOfMemory != 0)
25.     {
26.         num_ranks_ = (num_ranks_ > 0) ? num_ranks_ : 1;
27.         if (num_ranks_ == 0)
28.         {
29.             PRINT("WARNING: Cannot create memory system with "
30.                 << megsOfMemory << "MB, defaulting to minimum size of " << megsOfStoragePerRank
31.                 << "MB");
32.         }
33.     }
34.     setDevConfigParam(UINT, "NUM_RANKS", num_ranks_);
35.     // FIXME, need to synchronize between ConfigurationDB and Configuration
36.     config.NUM_RANKS = num_ranks_;
37. }
38. }
```

```
39. unsigned num_devices =
40.     (getConfigParam(UINT, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH"));
41. setDevConfigParam(UINT, "NUM_DEVICES", num_devices);
42. setDevConfigParam(UINT64, "TOTAL_STORAGE", num_ranks_ * Byte2MB(bytePerRank));
43.
44. PRINT("CH. " << systemID << " TOTAL_STORAGE : " << getConfigParam(UINT64, "TOTAL_STORAGE")
45.       << "MB | " << getConfigParam(UINT, "NUM_RANKS") << " Ranks | "
46.       << getConfigParam(UINT, "NUM_DEVICES") << " Devices per rank");
47.
48. memoryController = new MemoryController(this, csvOut, dramsimLog, config);
49.
50. // TODO: change to other vector constructor?
51. ranks = new vector<Rank*>();
52.
53. for (size_t i = 0; i < num_ranks_; i++)
54. {
55.     Rank* r = new Rank(dramsimLog, config);
56.     r->setChanId(systemID);
57.     r->setRankId(i);
58.     r->attachMemoryController(memoryController);
59.     r->pimRank->setChanId(systemID);
60.     r->pimRank->setRankId(i);
61.     ranks->push_back(r);
62. }
63.
64. memoryController->attachRanks(ranks);
65. }
```

MultiChannelMemorySystem.cpp

MemorySystem.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – SetUp()

- Memory 생성 시

- Rank개수만큼 Rank Class 생성 → Rank는 자신의 Class 변수에 channel, rank id 및 Bank들 관련 변수 초기화

```
1. MemorySystem::MemorySystem(unsigned id, unsigned int megsOfMemory, CSVWriter& csvOut,
2.   ostream& simLog, Configuration& configuration)
3. : dramsimLog(simLog),
4.   ReturnReadData(NULL),
5.   WriteDataDone(NULL),
6.   systemID(id),
7.   csvOut(csvOut),
8.   numOnTheFlyTransactions(0),
9.   config(configuration)
10. {
11.   currentClockCycle = 0;
12.
13.   DEBUG("==== MemorySystem " << systemID << " =====");
14.   uint64_t bytePerRank = static_cast<uint64_t>{
15.     (getConfigParam(UINT64, "NUM_ROWS") *
16.      getConfigParam(UINT, "NUM_COLS") * getConfigParam(UINT, "DEVICE_WIDTH")) *
17.     getConfigParam(UINT, "NUM_BANKS") *
18.     (getConfigParam(UINT64, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH")) /
19.     8);
20.   uint64_t megsOfStoragePerRank = bytePerRank >> 20; // bytes to MB (2^20 = 1,048,576 = Mega)
21.
22.   num_ranks_ = (megsOfMemory / Byte2MB(bytePerRank));
23.
24.   // If this is set, effectively override the number of ranks
25.   if (megsOfMemory != 0)
26.   {
27.     num_ranks_ = (num_ranks_ > 0) ? num_ranks_ : 1;
28.     if (num_ranks_ == 0)
29.     {
30.       PRINT("WARNING: Cannot create memory system with "
31.         << megsOfMemory << "MB, defaulting to minimum size of " << megsOfStoragePerRank
32.         << "MB");
33.     }
34.     setDevConfigParam(UINT, "NUM_RANKS", num_ranks_);
35.     // FIXME, need to synchronize between ConfigurationDB and Configuration
36.     config.NUM_RANKS = num_ranks_;
37.   }
38. }
```

MemorySystem.cpp

```
39. unsigned num_devices =
40.   (getConfigParam(UINT, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH"));
41. setDevConfigParam(UINT, "NUM_DEVICES", num_devices);
42. setDevConfigParam(UINT64, "TOTAL_STORAGE", num_ranks_ * Byte2MB(bytePerRank));
43.
44. PRINT("CH. " << systemID << " TOTAL_STORAGE : " << getConfigParam(UINT64, "TOTAL_STORAGE")
45.   << "MB | " << getConfigParam(UINT, "NUM_RANKS") << " Ranks | "
46.   << getConfigParam(UINT, "NUM_DEVICES") << " Devices per rank");
47.
48. memoryController = new MemoryController(this, csvOut, dramsimLog, config);
49.
50. // TODO: change to other vector constructor?
51. ranks = new vector<Rank*>();
52.
53. for (size_t i = 0; i < num_ranks_; i++)
54. {
55.   Rank* r = new Rank(dramsLog, config);
56.   r->setChanId(systemID);
57.   r->setRankId(i);
58.   r->attachMemoryController(memoryController);
59.   r->pimRank->setChanId(systemID);
60.   r->pimRank->setRankId(i);
61.   ranks->push_back(r);
62. }
63.
64. memoryController->attachRanks(ranks);
65. }
```

```
1. Rank::Rank(ostream& simLog, Configuration& configuration)
2. : chanId(-1),
3.   rankId(-1),
4.   dramsimLog(simLog),
5.   isPowerDown(false),
6.   refreshWaiting(false),
7.   readReturnCountdown(0),
8.   banks(getConfigParam(UINT, "NUM_BANKS"), Bank(simLog)),
9.   bankStates(getConfigParam(UINT, "NUM_BANKS"), BankState(simLog)),
10.  config(configuration),
11.  outgoingDataPacket(NULL),
12.  dataCyclesLeft(0),
13.  mode_(dramMode::SB)
14. {
15. {
16.   memoryController = NULL;
17.   currentClockCycle = 0;
18.   abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = sbmr1_ = sbmr2_ = false;
19.
20.   pimRank = make_shared<PIMRank>(dramsLog, config);
21.   pimRank->attachRank(this);
22. }
```

Rank.cpp

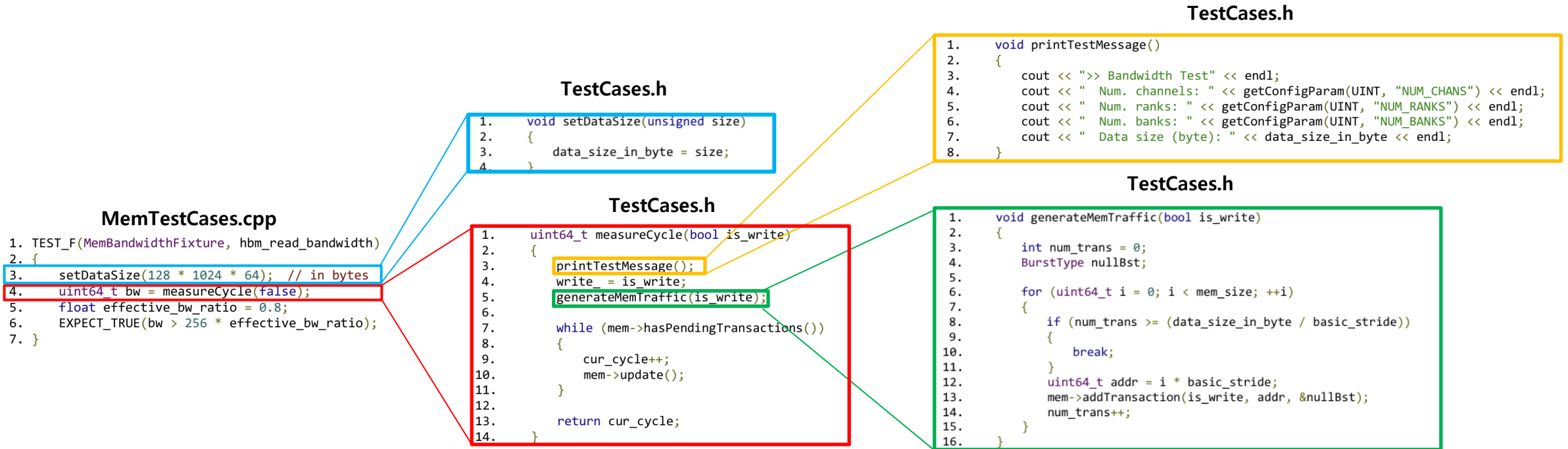
```
1. class Rank : public SimulatorObject
2. {
3. private:
4.   int chanId;
5.   int rankId;
6.   ostream& dramsimLog;
7.   bool isPowerDown;
8.   Configuration& config;
9.
10. public:
11.   // functions
12.   Rank(ostream& simLog, Configuration& configuration);
13.   virtual ~Rank();
14.
15.   void receiveFromBus(BusPacket* packet);
16.   void check(BusPacket* packet);
17.   void updateState(BusPacket* packet);
18.   void sendToBank(BusPacket* packet);
19.
20.   void checkBank(BusPacketType type, int bank, int row);
21.   void updateBank(BusPacketType type, int bank, int row, bool targetBank, bool targetBankgroup);
22.   void attachMemoryController(MemoryController* mc);
23.   int getChanId() const;
24.   void setChanId(int id);
25.   int getRankId() const;
26.   void setRankId(int id);
27.   void update();
28.   void powerUp();
29.   void powerDown();
30.
31.   void readSb(BusPacket* packet);
32.   void writeSb(BusPacket* packet);
33.
34. // fields
35. MemoryController* memoryController;
36. BusPacket* outgoingDataPacket;
37. shared_ptr<PIMRank> pimRank;
38. unsigned dataCyclesLeft;
39. bool refreshWaiting;
40.
41. // these are vectors so that each element is per-bank
42. vector<BusPacket*> readReturnPacket;
43. vector<unsigned> readReturnCountdown;
44.
45. vector<Bank> banks;
46. vector<BankState> bankStates;
47.
48. dramMode mode_;
49. bool abmr1Even_, abmr1Odd_, abmr2Even_, abmr2Odd_, sbmr1_, sbmr2_;
50.
51. const char* getModeColor()
52. {
53.   switch (mode_)
54.   {
55.     case dramMode::SB:
56.       return END;
57.     case dramMode::HAB:
58.       return GREEN;
59.     case dramMode::HAB_PIM:
60.       return CYAN;
61.   }
62.   return GRAY;
63. }
64. ;;
```

Rank.h

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- mem 초기화 이후 data_size_in_byte를 설정
- **measureCycle()**을 통하여 성능을 simulate
- **generateMemTraffic()**은 memory transaction을 생성하는 함수 → Memory Trace를 생성하는 역할과 동일



Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle() → generateMemTraffic()

- **generateMemTraffic()**은 어떤, 얼마만큼의 memory transaction(Read or Write)을 수행할 것인지 결정하여 transactionQueue를 채움
- transactionQueue는 최종적으로 MemoryController.cpp에서 관리하며, scheduling logic에 따라 issue됨
- **MultiChannelMemorySystem::findChannelNumber()**은 다음 page에서 설명
- mem_size 및 basic_stride는 PPT page7의 SetUp()에서 정의됨
 - $\text{mem_size} = \text{NUM_CHANS} \times \text{NUM_RANKS} \times \text{NUM_BANK_GROUPS} \times \text{NUM_BANKS} \times \text{NUM_ROWS} \times \text{NUM_COLS}$
 - $\text{basic_stride} = (\text{JEDEC_DATA_BUS_BITS} \times \text{BL}) / 8$ [ex. JEDEC_DATA_BUS_BITS = 64, BL = 4, basic_stride = (64 bits × 4) / 8 bytes/bit = 32 bytes (= 한 번의 READ/WRITE transaction 시 전송되는 data 크기)]

TestCases.h

```
1. void generateMemTraffic(bool is_write)
2. {
3.     int num_trans = 0;
4.     BurstType nullBst;
5.
6.     for (uint64_t i = 0; i < mem_size; ++i)
7.     {
8.         if (num_trans >= (data_size_in_byte / basic_stride))
9.         {
10.            break;
11.        }
12.        uint64_t addr = i * basic_stride;
13.        mem->addTransaction(is_write, addr, &nullBst);
14.        num_trans++;
15.    }
16. }
```

MultiChannelMemorySystem.cpp

```
1. bool MultiChannelMemorySystem::addTransaction(bool isWrite, uint64_t addr, BurstType* data)
2. {
3.     unsigned channelNumber = findChannelNumber(addr);
4.     return channels[channelNumber]->addTransaction(isWrite, addr, data);
5. }
```

```
1. unsigned MultiChannelMemorySystem::findChannelNumber(uint64_t addr)
2. {
3.     // Single channel case is a trivial shortcut case
4.     if (configuration->NUM_CHANS == 1)
5.     {
6.         return 0;
7.     }
8.
9.     if (!isPowerOfTwo(configuration->NUM_CHANS))
10.    {
11.        ERROR("We can only support power of two # of channels.\n"
12.            << "I don't know what Intel was thinking, but trying to address map half"
13.            << " a bit is a neat trick that we're not sure how to do");
14.        abort();
15.    }
16.
17.    // only chan is used from this set
18.    unsigned channelNumber, rank, bank, row, col;
19.    addrMapping->addressMapping(addr, channelNumber, rank, bank, row, col);
20.    if (channelNumber >= configuration->NUM_CHANS)
21.    {
22.        ERROR("Got channel index " << channelNumber << " but only " << configuration->NUM_CHANS
23.            << " exist");
24.        abort();
25.    }
26.    return channelNumber;
27. }
```

MultiChannelMemorySystem.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle() → generateMemTraffic()

- **addressMapping**의 경우 주어진 address를 shifting 및 slicing함으로써 처리됨
- **addressMapping**에서 address의 하위 6bit(= transaction size 및 burst size 나타내는 bit)는 shift right하여 제거해야 함
 - Burst: 한 번에 읽는 작은 데이터 덩어리 (DDR에서 보통 8byte), Transaction: 전체 요청 단위, 여러 burst로 구성, 보통 cache line 크기와 동일(ex. 64byte)
 - Controller는 Transaction 1개를 받아서 처리 & Column Address는 Burst 중 Byte마다 +1 증가 → 따라서, if transaction = 64byte이면, 64byte = 2^6 byte → 6 bit right shift 필요
- Shift하고 남은 bit들은 적절히 slicing하여 scheme에 따라, (channel : rank : bank : row : col)로 분리 (scheme마다 slice방법 다름)
- **MultiChannelMemorySystem::findChannelNumber()**는 channelNumber(= channel id)를 반환

MultiChannelMemorySystem.cpp

```
1. unsigned MultiChannelMemorySystem::findChannelNumber(uint64_t addr)
2. {
3.     // Single channel case is a trivial shortcut case
4.     if (configuration->NUM_CHANS == 1)
5.     {
6.         return 0;
7.     }
8.
9.     if (!isPowerOfTwo(configuration->NUM_CHANS))
10.    {
11.        ERROR("We can only support power of two # of channels.\n")
12.        << "I don't know what Intel was thinking, but trying to address map half"
13.        << " a bit is a neat trick that we're not sure how to do";
14.        abort();
15.    }
16.
17.    // only chan is used from this set
18.    unsigned channelNumber, rank, bank, row, col;
19.    addrMapping->addressMapping(addr, channelNumber, rank, bank, row, col);
20.    if (channelNumber >= configuration->NUM_CHANS)
21.    {
22.        ERROR("Got channel index " << channelNumber << " but only " << configuration->NUM_CHANS
23.        << " exist");
24.        abort();
25.    }
26.    return channelNumber;
27. }
```

addressMapping.cpp

```
1. void AddrMapping::addressMapping(uint64_t physicalAddress, unsigned& newTransactionChan,
2. unsigned& newTransactionRank, unsigned& newTransactionBank,
3. unsigned& newTransactionRow, unsigned& newTransactionColumn)
4. {
5.     if ((physicalAddress & transactionMask) != 0)
6.     {
7.         DEBUG("WARNING: address 0x" << std::hex << physicalAddress << std::dec
8.         << " is not aligned to the request size of "
9.         << transactionSize);
10.    }
11.    // each burst will contain JEDEC_DATA_BUS_BITS/8 bytes of data, so the bottom
12.    // bits (3 bits for a single channel DDR system) are thrown away before mapping the other bits
13.    physicalAddress >>= byteOffsetWidth;
14.
15.    // The next thing we have to consider is that when a request is made for a
16.    // we've taken into account the granularity of a single burst by shifting
17.    // off the bottom 3 bits, but a transaction has to take into account the
18.    // burst length (i.e. the requests will be aligned to cache line sizes which
19.    // should be equal to transactionSize above).
20.    // Since the column address increments internally on bursts, the bottom n
21.    // bits of the column (colLow) have to be zero in order to account for the
22.    // total size of the transaction. These n bits should be shifted off the
23.    // address and also subtracted from the total column width.
24.    // zero. These zero bits must be made up of the byte offset (3 bits) and
25.    // also
26.    // from the bottom bits of the column
27.    // For example: colLowBits = log2(64bytes) - 3 bits = 3 bits
28.    physicalAddress >>= colLowBitWidth;
29.
30.    if (DEBUG_ADDR_MAP)
31.    {
32.        DEBUG("Bit widths: ch:" << channelBitWidth << " r:" << rankBitWidth << " b:" << bankBitWidth
33.        << " row:" << rowBitWidth << " colLow:" << colLowBitWidth
34.        << " colHigh:" << colHighBitWidth << " off:" << byteOffsetWidth
35.        << " Total:"
36.        << "(channelBitWidth + rankBitWidth + bankBitWidth + rowBitWidth +
37.        colLowBitWidth + colHighBitWidth + byteOffsetWidth)");
38.    }
39.
40.    // perform various address mapping schemes
41.    if (addressMappingScheme == Scheme1)
42.    {
43.        // chan:rank:row:col:bank
44.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
45.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
46.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
47.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
48.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
49.    }
50.    else if (addressMappingScheme == Scheme2)
51.    {
52.        // chan:row:col:bank:rank
53.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
54.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
55.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
56.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
57.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
58.    }
59.    else if (addressMappingScheme == Scheme3)
60.    {
61.        // chan:rank:bank:col:row
62.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
63.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
64.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
65.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
66.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
67.    }
68.    else if (addressMappingScheme == Scheme4)
69.    {
70.        // chan:rank:bank:row:col
71.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
72.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
73.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
74.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
75.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
76.    }
77.    else if (addressMappingScheme == Scheme5)
78.    {
79.        // chan:row:col:rank:bank
80.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
81.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
82.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
83.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
84.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
85.    }
86.    else if (addressMappingScheme == Scheme6)
87.    {
88.        // chan:row:bank:rank:col
89.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
90.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
91.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
92.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
93.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
94.    }
95.    else if (addressMappingScheme == Scheme7)
96.    {
97.        // row:col:rank:bank:chan
98.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
99.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
100.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
101.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
102.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
103.    }
104.    else if (addressMappingScheme == Scheme8)
105.    {
106.        // row:col:rank:bank:chan
107.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
108.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
109.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
110.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
111.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
112.    }
113.    else if (addressMappingScheme == Scheme9)
114.    {
115.        // row:col:rank:bank:chan
116.        newTransactionChan = diffBitWidth(physicalAddress, channelBitWidth);
117.        newTransactionBank = diffBitWidth(physicalAddress, bankBitWidth);
118.        newTransactionRank = diffBitWidth(physicalAddress, rankBitWidth);
119.        newTransactionColumn = diffBitWidth(physicalAddress, colHighBitWidth);
120.        newTransactionRow = diffBitWidth(physicalAddress, rowBitWidth);
121.    }
122.    else
123.    {
124.        ERROR("Error - Unknown Address Mapping Scheme");
125.        exit(-1);
126.    }
127.
128.    if (DEBUG_ADDR_MAP)
129.    {
130.        DEBUG("Mapped Ch=" << newTransactionChan << " Rank=" << newTransactionRank
131.        << " Bank=" << newTransactionBank << " Row=" << newTransactionRow
132.        << " Col=" << newTransactionColumn << "\n");
133.    }
134. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle() → generateMemTraffic()

- **addTransaction()**은 해당 transaction의 addr가 나타내는 **channel의 Memory Controller**에게 해당 transaction을 넘김
- Memory Controller는 **transactionQueue**가 **꽉 차지 않았으면**, 해당 transaction을 Queue에 저장
- 꽉 찼으면 pendingTransactions 배열에 저장됨
- **trans->timeAdded**는 기존 DRAMSim2에서 latency 지표 계산에 쓰임
 - But, DRAMSim2에서만 **trans->timeAdded**가 statistic에 반영되고, PIMSimulator는 이를 반영하지 않음 (사실상 안 쓰이는 변수)

MultiChannelMemorySystem.cpp

```
1. bool MultiChannelMemorySystem::addTransaction(bool isWrite, uint64_t addr, BurstType* data)
2. {
3.     unsigned channelNumber = findChannelNumber(addr);
4.     return channels[channelNumber]->addTransaction(isWrite, addr, data);
5. }
```

MemorySystem.cpp

```
1. bool MemorySystem::addTransaction(bool isWrite, uint64_t addr, BurstType* data)
2. {
3.     TransactionType type = isWrite ? DATA_WRITE : DATA_READ;
4.     Transaction* trans = new Transaction(type, addr, data);
5.     return addTransaction(trans);
6. }
```

MemorySystem.cpp

```
1. bool MemorySystem::addTransaction(Transaction* trans)
2. {
3.     if (memoryController->WillAcceptTransaction())
4.     {
5.         return memoryController->addTransaction(trans);
6.     }
7.     else
8.     {
9.         pendingTransactions.push_back(trans);
10.        return true;
11.    }
12. }
```

MemoryController.cpp

```
1. bool MemoryController::WillAcceptTransaction()
2. {
3.     return transactionQueue.size() < getConfigParam(UINT, "TRANS_QUEUE_DEPTH");
4. }
```

MemoryController.cpp

```
1. bool MemoryController::addTransaction(Transaction* trans)
2. {
3.     if (WillAcceptTransaction())
4.     {
5.         parentMemorySystem->numOnTheFlyTransactions++;
6.         trans->timeAdded = currentClockCycle;
7.         transactionQueue.push_back(trans);
8.         return true;
9.     }
10.    else
11.    {
12.        return false;
13.    }
14. }
```


Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle() → update()

- **measureCycle()**을 통하여 성능을 simulate
- CPU clock과 DRAM clock의 **clock domain**을 정렬하며, **callback**을 호출
- 등록된 **callback**은 **actual_update()** 임 (PPT page7 참고)

TestCases.h

```
1. uint64_t measureCycle(bool is_write)
2. {
3.     printTestMessage();
4.     write_ = is_write;
5.     generateMemTraffic(is_write);
6.
7.     while (mem->hasPendingTransactions())
8.     {
9.         cur_cycle++;
10.        mem->update();
11.    }
12.
13.    return cur_cycle;
14. }
```

```
1. void MultiChannelMemorySystem::update()
2. {
3.     clockDomainCrossover.update();
4. }
5.
6. void MultiChannelMemorySystem::actual_update()
7. {
8.     if (currentClockCycle == 0)
9.     {
10.        InitOutputFiles(traceFilename);
11.        DEBUG("DRAMSim2 Clock Frequency = " << clockDomainCrossover.clock1
12.              << "Hz, CPU Clock Frequency="
13.              << clockDomainCrossover.clock2 << "Hz");
14.    }
15.
16.    if (currentClockCycle > 0 && currentClockCycle % configuration->EPOCH_LENGTH == 0)
17.    {
18.        (*csvOut) << "ms" << currentClockCycle * configuration->tCK * 1E-6;
19.        for (size_t i = 0; i < configuration->NUM_CHANS; i++)
20.        {
21.            channels[i]->printStats(false);
22.        }
23.        csvOut->finalize();
24.    }
25.
26.    for (size_t i = 0; i < configuration->NUM_CHANS; i++)
27.    {
28.        channels[i]->update();
29.    }
30.
31.    currentClockCycle++;
32. }
```

MultiChannelMemorySystem.cpp

```
1. void MultiChannelMemorySystem::setCPUClockSpeed(uint64_t cpuClkFreqHz)
2. {
3.     uint64_t dramsimClkFreqHz = (uint64_t)(1.0 / (configuration->tCK * 1e-9));
4.     clockDomainCrossover.clock1 = dramsimClkFreqHz; // ← DRAM 클럭 (Hz)
5.     clockDomainCrossover.clock2 = cpuClkFreqHz; // ← CPU 클럭 (Hz)
6. }
```

```
1. namespace ClockDomain
2. {
3.     // "Default" crosser with a 1:1 ratio
4.     ClockDomainCrossover::ClockDomainCrossover(ClockUpdateCB* _callback)
5.         : callback(_callback), clock1(1UL), clock2(1UL), counter1(0UL), counter2(0UL)
6. }
```

```
1. void ClockDomainCrossover::update()
2. {
3.     // short circuit case for 1:1 ratios
4.     if (clock1 == clock2 && callback)
5.     {
6.         (*callback)();
7.         return;
8.     }
9.
10.    // Update counter 1.
11.    counter1 += clock1;
12.
13.    while (counter2 < counter1)
14.    {
15.        counter2 += clock2;
16.        // PRINT("CALLBACK: counter1= " << counter1 << "; counter2= " << counter2/ << "; ");
17.        // PRINT("callback address: " << (uint64_t)callback);
18.        if (callback)
19.        {
20.            // PRINT("Callback() " << (uint64_t)callback << "Counters:1=" << counter1 << ",
21.            // 2=" << counter2);
22.            (*callback)();
23.        }
24.    }
25.
26.    if (counter1 == counter2)
27.    {
28.        counter1 = 0;
29.        counter2 = 0;
30.    }
31. }
```

ClockDomain.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → Rank::update()
- Rank::update() – 특이사항: 현재 Simulator는 독립적인 command/read/write bus를 사용하고 있음
 1. Read Latency 대기 (ex. config.RL = 20 → 20 cycle 기다린 후 data를 bus에 올림)
 2. Data bus timing 관리 (= BL/2 설정) → Bus에 올라간 data는 BL/2 cycle간 transfer
 3. 실제로 BL/2 cycle만큼 대기 후 완료 시 memoryController에게 완료를 알람

```
6. void MultiChannelMemorySystem::actual_update()
7. {
8.     if (currentClockCycle == 0)
9.     {
10.         InitOutputFiles(traceFilename);
11.         DEBUG("DRAMSim2 Clock Frequency =" << clockDomainCrosser.clock1
12.             << "Hz, CPU Clock Frequency="
13.             << clockDomainCrosser.clock2 << "Hz");
14.     }
15.
16.     if (currentClockCycle > 0 && currentClockCycle % configuration->EPOCH_LENGTH == 0)
17.     {
18.         (*csvOut) << "ms" << currentClockCycle * configuration->tCK * 1E-6;
19.         for (size_t i = 0; i < configuration->NUM_CHANS; i++)
20.         {
21.             channels[i]->printStats(false);
22.         }
23.         csvOut->finalize();
24.     }
25.
26.     for (size_t i = 0; i < configuration->NUM_CHANS; i++)
27.     {
28.         channels[i]->update();
29.     }
30.
31.     currentClockCycle++;
32. }
```

MultiChannelMemorySystem.cpp

```
1. // update the memory systems state
2. void MemorySystem::update()
3. {
4.     // PRINT(" ----- Memory System Update -----");
5.
6.     // updates the state of each of the objects
7.     // NOTE - do not change order
8.     for (size_t i = 0; i < num_ranks_; i++)
9.     {
10.         (*ranks)[i]->update();
11.     }
12. }
13.
14. void MemoryController::receiveFromBus(BusPacket* bpacket)
15. {
16.     if (bpacket->busPacketType != DATA)
17.     {
18.         ERROR("== Error - Memory Controller received a non-DATA bus packet from rank");
19.         bpacket->print();
20.         exit(0);
21.     }
22.
23.     if (DEBUG_BUS)
24.     {
25.         PRINTN(" -- MC Receiving From Data Bus : ");
26.         bpacket->print();
27.     }
28.
29.     // add to return read data queue
30.     returnTransaction.push_back(
31.         new Transaction(RETURN_DATA, bpacket->physicalAddress, bpacket->data));
32.
33.     // this delete statement saves a mindboggling amount of memory
34.     delete (bpacket);
35. }
```

```
1. void Rank::update()
2. {
3.     // An outgoing packet is one that is currently sending on the bus
4.     // do the book keeping for the packet's time left on the bus
5.     if (outgoingDataPacket != NULL)
6.     {
7.         dataCyclesLeft--;
8.         if (dataCyclesLeft == 0)
9.         {
10.            // if the packet is done on the bus, call receiveFromBus and free up
11.            // the bus
12.            memoryController->receiveFromBus(outgoingDataPacket);
13.            outgoingDataPacket = NULL;
14.        }
15.    }
16.
17.    // decrement the counter for all packets waiting to be sent back
18.    for (size_t i = 0; i < readReturnCountdown.size(); i++) readReturnCountdown[i]--;
19.
20.    if (readReturnCountdown.size() > 0 && readReturnCountdown[0] == 0)
21.    {
22.        // RL time has passed since the read was issued; this packet is
23.        // ready to go out on the bus
24.
25.        outgoingDataPacket = readReturnPacket[0];
26.        dataCyclesLeft = config.BL / 2;
27.
28.        // remove the packet from the ranks
29.        readReturnPacket.erase(readReturnPacket.begin());
30.        readReturnCountdown.erase(readReturnCountdown.begin());
31.
32.        if (DEBUG_BUS)
33.        {
34.            PRINTN(" -- R" << getChanId() << " Issuing On Data Bus : ");
35.            outgoingDataPacket->print();
36.            PRINT("");
37.        }
38.    }
39.    pimRank->update();
40.    pimRank->step();
41. }
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()
- Transaction Queue에 미처 올리지 못한 transaction을 올릴 수 있는 지 확인
- 그 후, memoryController::update()를 호출
- 다음 page에서 memoryController::update() 동작을 분석

```
1. // update the memory systems state
2. void MemorySystem::update()
3. {
4.     // PRINT(" ----- Memory System Update -----");
5.
6.     // updates the state of each of the objects
7.     // NOTE - do not change order
8.     for (size_t i = 0; i < num_ranks_; i++)
9.     {
10.        (*ranks)[i] -> update();
11.    }
12.
13.    // pendingTransactions will only have stuff in it if MARSS is adding stuff
14.    if (pendingTransactions.size() > 0 && memoryController->WillAcceptTransaction())
15.    {
16.        memoryController->addTransaction(pendingTransactions.front());
17.        pendingTransactions.pop_front();
18.    }
19.    memoryController->update();
20.
21.    // simply increments the currentClockCycle field for each object
22.    for (size_t i = 0; i < num_ranks_; i++)
23.    {
24.        (*ranks)[i] -> step();
25.    }
26.    memoryController->step();
27.    this->step();
28.
29.    // PRINT("\n"); // two new lines
30. }
```

MemorySystem.cpp

```
1. void MemoryController::update()
2. {
3.     updateBankState();
4.     // check for outgoing command packets and handle countdowns
5.     if (outgoingCmdPacket != NULL)
6.     {
7.         cmdCyclesLeft--;
8.         if (cmdCyclesLeft == 0) // packet is ready to be received by rank
9.         {
10.            (*ranks)[outgoingCmdPacket->rank] -> receiveFromBus(outgoingCmdPacket);
11.            outgoingCmdPacket = NULL;
12.        }
13.    }
14.
15.    // check for outgoing data packets and handle countdowns
16.    if (outgoingDataPacket != NULL)
17.    {
18.        dataCyclesLeft--;
19.        if (dataCyclesLeft == 0)
20.        {
21.            // inform upper levels that a write is done
22.            if (parentMemorySystem->WriteDataDone != NULL)
23.            {
24.                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
25.                outgoingDataPacket->physicalAddress,
26.                currentClockCycle);
27.            }
28.            parentMemorySystem->numOnTheFlyTransactions--;
29.            (*ranks)[outgoingDataPacket->rank] -> receiveFromBus(outgoingDataPacket);
30.            outgoingDataPacket = NULL;
31.        }
32.    }
33. }
```

```
34. // if any outstanding write data needs to be sent
35. // and the appropriate amount of time has passed (WL)
36. // then send data on bus
37. //
38. // write data held in fifo vector along with countdowns
39. if (writeDataCountdown.size() > 0)
40. {
41.     for (size_t i = 0; i < writeDataCountdown.size(); i++) writeDataCountdown[i]--;
42.
43.     if (writeDataCountdown[0] == 0)
44.     {
45.         // send to bus and print debug stuff
46.         if (DEBUG_BUS)
47.         {
48.             PRINTN(" -- MC Issuing On Data Bus : ");
49.             writeDataToSend[0] -> print();
50.         }
51.
52.         // queue up the packet to be sent
53.         if (outgoingDataPacket != NULL)
54.         {
55.             ERROR("Error - Data Bus Collision");
56.             exit(-1);
57.         }
58.
59.         outgoingDataPacket = writeDataToSend[0];
60.         dataCyclesLeft = config.BL / 2;
61.
62.         totalTransactions++;
63.
64.         writeDataCountdown.erase(writeDataCountdown.begin());
65.         writeDataToSend.erase(writeDataToSend.begin());
66.     }
67. }
68.
69. // if its time for a refresh issue a refresh
70. // else pop from command queue if it's not empty
71. updateRefresh();
72.
73. // pass a pointer to a poppedBusPacket
74. // function returns true if there is something valid in poppedBusPacket
75. if (commandQueue.pop(&poppedBusPacket))
76. {
77.     updateCommandQueue(poppedBusPacket);
78. }
79. }
```

```
80. updateTransactionQueue();
81.
82. // check for outstanding data to return to the CPU
83. if (returnTransaction.size() > 0)
84. {
85.     if (DEBUG_BUS)
86.     {
87.         PRINTN(" -- MC Issuing to CPU bus : " << *returnTransaction[0]);
88.         totalTransactions++;
89.     }
90.     bool foundMatch = false;
91.     // find the pending read transaction to calculate latency
92.     for (size_t i = 0; i < pendingReadTransactions.size(); i++)
93.     {
94.         if (pendingReadTransactions[i] -> address == returnTransaction[0] -> address)
95.         {
96.             unsigned chan, rank, bank, row, col;
97.             config.addrMapping.addressMapping(returnTransaction[0] -> address, chan, rank, bank,
98.             row, col);
99.             memoryContStats->insertHistogram(
100.             currentClockCycle - pendingReadTransactions[i] -> timeAdded, rank, bank);
101.             // FIXME: Is it correct?
102.             // memcpy(pendingReadTransactions[i] -> data,
103.             // returnTransaction[0] -> data, config.BL * (JEDEC_DATA_BUS_BITS / 8));
104.             returnReadData(pendingReadTransactions[i]);
105.
106.             delete pendingReadTransactions[i];
107.             pendingReadTransactions.erase(pendingReadTransactions.begin() + i);
108.             foundMatch = true;
109.             break;
110.         }
111.     }
112.     if (!foundMatch)
113.     {
114.         ERROR("Can't find a matching transaction for 0x" << hex << returnTransaction[0] -> address
115.         << dec);
116.         abort();
117.     }
118.     delete returnTransaction[0];
119.     returnTransaction.erase(returnTransaction.begin());
120. }
121.
122. // decrement refresh counters
123. for (size_t i = 0; i < config.NUM_RANKS; i++) refreshCountdown[i]--;
124. for (size_t i = 0; i < config.NUM_BANKS; i++) refreshCountdownBank[i]--;
125.
126. // print debug
127. printDebugOnUpdate();
128. commandQueue.step();
129. }
```

MemoryController.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()

1. DRAM Bank들의 state 변화가 완료되었는지 확인

- 모든 Rank와 Bank를 순회
- Count down: 이전 cycle에서 PRECHARGE나 REFRESH 명령이 수행되었다면, 해당 명령의 완료 시간(ex. tRP, tRFC)만큼 counter가 설정되어 있음
- State transition: 매 cycle 1씩 감소하다가 0이 되면, 해당 Bank의 상태를 Idle로 변경하여 다음 명령(ex. ACTIVATE)을 받을 수 있는 상태로 만듦

```
1. void MemoryController::update()
2. {
3.     updateBankState();
4.     // check for outgoing command packets and handle countdowns
5.     if (outgoingCmdPacket != NULL)
6.     {
7.         cmdCyclesLeft--;
8.         if (cmdCyclesLeft == 0) // packet is ready to be received by rank
9.         {
10.            (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
11.            outgoingCmdPacket = NULL;
12.        }
13.    }
14.    // check for outgoing data packets and handle countdowns
15.    if (outgoingDataPacket != NULL)
16.    {
17.        dataCyclesLeft--;
18.        if (dataCyclesLeft == 0)
19.        {
20.            // inform upper levels that a write is done
21.            if (parentMemorySystem->WriteDataDone != NULL)
22.            {
23.                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
24.                outgoingDataPacket->physicalAddress,
25.                currentClockCycle);
26.            }
27.            parentMemorySystem->numOnTheFlyTransactions--;
28.            (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
29.            outgoingDataPacket = NULL;
30.        }
31.    }
32. }
33.
```

MemoryController.cpp

```
1. void MemoryController::updateBankState()
2. {
3.     for (size_t i = 0; i < config.NUM_RANKS; i++)
4.     {
5.         for (size_t j = 0; j < config.NUM_BANKS; j++)
6.         {
7.             if (bankStates[i][j].stateChangeCountdown > 0)
8.             {
9.                 // decrement counters
10.                bankStates[i][j].stateChangeCountdown--;
11.            }
12.            // if counter has reached 0, change state
13.            if (bankStates[i][j].stateChangeCountdown == 0)
14.            {
15.                switch (bankStates[i][j].lastCommand)
16.                {
17.                    case REF:
18.                    case RFCSB:
19.                    case PRECHARGE:
20.                        bankStates[i][j].currentBankState = Idle;
21.                        break;
22.                    default:
23.                        break;
24.                }
25.            }
26.        }
27.    }
28. }
29.
```

MemoryController.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()

2. Command Bus

- DRAM의 Command Bus에 실린 packet이 물리적으로 Rank에 도달하는데 걸리는 cycle을 count
- outgoingCmdPacket: 현재 Command Bus에 있는 command packet → cmdCyclesLeft: bus transfer latency(ex. tCMD cycle)을 추적
- cmdCyclesLeft==0 시, Rank에 도착한 것이기에 ranks[...]→receiveFromBus()를 호출 → outgoingCmdPacket = NULL로 초기화

```
1. void MemoryController::update()
2. {
3.     updateBankState();
4.     // check for outgoing command packets and handle countdowns
5.     if (outgoingCmdPacket != NULL)
6.     {
7.         cmdCyclesLeft--;
8.         if (cmdCyclesLeft == 0) // packet is ready to be received by rank
9.         {
10.            (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
11.            outgoingCmdPacket = NULL;
12.        }
13.    }
14.
15.    // check for outgoing data packets and handle countdowns
16.    if (outgoingDataPacket != NULL)
17.    {
18.        dataCyclesLeft--;
19.        if (dataCyclesLeft == 0)
20.        {
21.            // inform upper levels that a write is done
22.            if (parentMemorySystem->WriteDataDone != NULL)
23.            {
24.                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
25.                outgoingDataPacket->physicalAddress,
26.                currentClockCycle);
27.            }
28.            parentMemorySystem->numOnTheFlyTransactions--;
29.            (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
30.            outgoingDataPacket = NULL;
31.        }
32.    }
33. }
```

MemoryController.cpp

```
1. void Rank::receiveFromBus(BusPacket* packet)
2. {
3.     if (DEBUG_BUS)
4.     {
5.         PRINTN(" -- R" << getChanId() << " Receiving On Bus   : ");
6.         packet->print();
7.     }
8.     if (VERIFICATION_OUTPUT)
9.     {
10.        packet->print(currentClockCycle, false);
11.    }
12.    if (!pimRank->isReservedRA(packet->row))
13.    {
14.        check(packet);
15.        updateState(packet);
16.    }
17.    sendToBank(packet);
18. }
```

Rank.cpp

[cf] packet 구성

```
cmdPacket
패킷 구성:
busPacketType: ACTIVATE
physicalAddress: 0x12345600
rank: 0
bank: 2
row: 1024
column: (사용 안 함)
data: NULL

WRITE 를 위한 DataPacket
패킷 구성:
busPacketType: DATA
physicalAddress: 0x12345600
rank: 0
bank: 2
row: 1024
column: 8
data: [0xAF, 0xBE, 0xAD, ...] (64 바이트 분량의 실제 값) or NULL
```

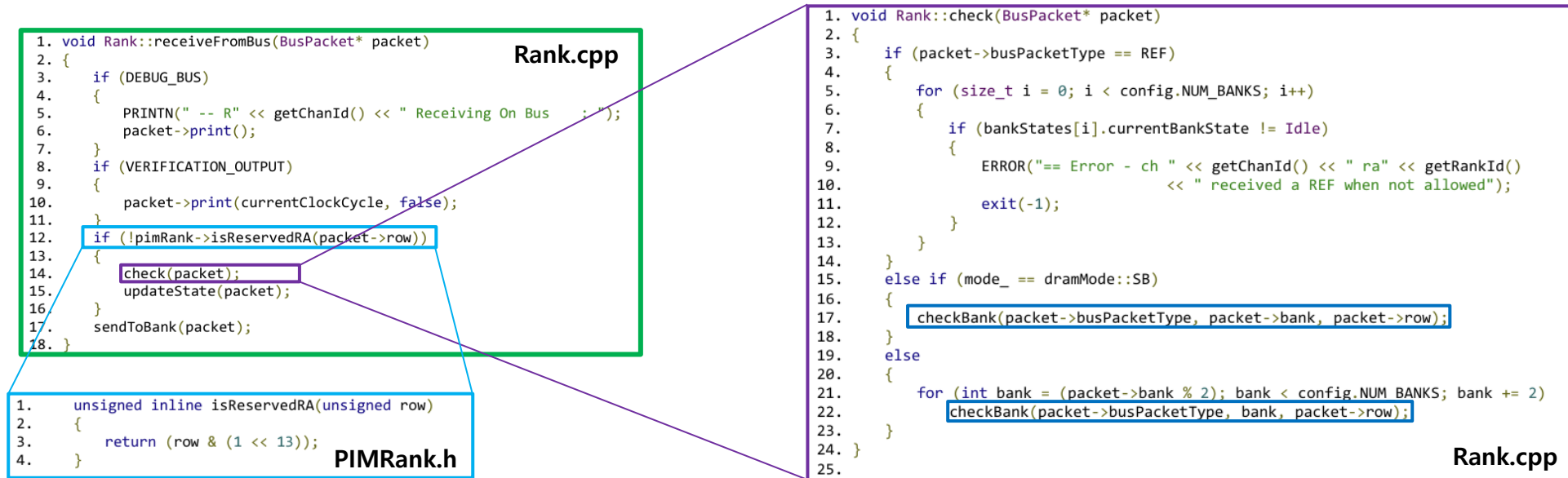
Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → **MemoryController::update()** → **Rank::receiveFromBus()** → **Rank::check()**

2. Command Bus → **receiveFromBus()** → **check()**

- 명령이 PIM 연산을 위한 Reserved Row Address를 target하는지 확인 → 일반 DRAM 동작이 아닌 특수 mode라면 표준 timing check를 건너뛴
- 현재 Bank State에서 이 명령이 실행 가능한지 검사 → 다음 page에 계속



Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → **MemoryController::update()** → **Rank::receiveFromBus()** → **Rank::check()**
 - 현재 Bank State에서 이 명령이 실행 가능한지 검사
 - REF: 모든 Bank가 Idle 상태(open row가 없는 상태) → RowActive 상태인 Bank가 있다면 REFRESH 허용 X
 - Single Bank mode(= 일반적인 DRAM 동작) 검사 → packet에 명시된 단 1개의 Bank에 대해서만 timing을 check
 - HAB/PIM Mode 검사 (= Multi Bank Mode) → 여러 Bank를 동시에 제어 (모든 짝수 Bank 또는 모든 홀수 Bank를 순회)

```
1. void Rank::check(BusPacket* packet)
2. {
3.     if (packet->busPacketType == REF)
4.     {
5.         for (size_t i = 0; i < config.NUM_BANKS; i++)
6.         {
7.             if (bankStates[i].currentBankState != Idle)
8.             {
9.                 ERROR("== Error - ch " << getChanId() << " ra" << getRankId()
10.                    << " received a REF when not allowed");
11.                 exit(-1);
12.             }
13.         }
14.     }
15.     else if (mode_ == dramMode::SB)
16.     {
17.         checkBank(packet->busPacketType, packet->bank, packet->row);
18.     }
19.     else
20.     {
21.         for (int bank = (packet->bank % 2); bank < config.NUM_BANKS; bank += 2)
22.             checkBank(packet->busPacketType, bank, packet->row);
23.     }
24. }
25.
```

Rank.cpp

```
1. void Rank::checkBank(BusPacketType type, int bank, int row)
2. {
3.     switch (type)
4.     {
5.         case READ:
6.             if (bankStates[bank].currentBankState != RowActive ||
7.                 currentClockCycle < bankStates[bank].nextRead ||
8.                 row != bankStates[bank].openRowAddress)
9.             {
10.                ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
11.                   << " received a READ when not allowed @ "
12.                   << currentClockCycle);
13.                exit(-1);
14.            }
15.            break;
16.         case WRITE:
17.             if (bankStates[bank].currentBankState != RowActive ||
18.                 currentClockCycle < bankStates[bank].nextWrite ||
19.                 row != bankStates[bank].openRowAddress)
20.             {
21.                ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
22.                   << " received a WRITE when not allowed @ "
23.                   << currentClockCycle);
24.                bankStates[bank].print();
25.                exit(-1);
26.            }
27.            break;
28.     }
29. }
```

```
29.     case ACTIVATE:
30.         if (bankStates[bank].currentBankState != Idle ||
31.             currentClockCycle < bankStates[bank].nextActivate)
32.         {
33.             ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
34.                << " received a ACT when not allowed @ "
35.                << currentClockCycle);
36.             bankStates[bank].print();
37.             exit(-1);
38.         }
39.         break;
40.     case PRECHARGE:
41.         if (bankStates[bank].currentBankState != RowActive ||
42.             currentClockCycle < bankStates[bank].nextPrecharge)
43.         {
44.             ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
45.                << " received a PRE when not allowed @ "
46.                << currentClockCycle);
47.             exit(-1);
48.         }
49.         break;
50.     case DATA:
51.         break;
52.     default:
53.         ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
54.         exit(-1);
55.         break;
56. }
57. }
58. }
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → **MemoryController::update()** → **Rank::receiveFromBus()** → **Rank::updateState()**

2. Command Bus → **receiveFromBus()** → **updateState()**

- updateState()** : command을 받을 수 있으므로, 다음 command가 언제 들어올 수 있는지 timing을 update (현재 command는 실행될 예정 → timing update 필요)

- updateBank()**

1. REFRESH 시, Refresh 중에는 ACTIVATE command를 issue 못하기에, ACTIVATE command를 언제부터 issue가능한지 timing update

- 다음 page에 계속

```
1. void Rank::receiveFromBus(BusPacket* packet)
2. {
3.     if (DEBUG_BUS)
4.     {
5.         PRINTN(" -- R" << getChanId() << " Receiving On Bus : ");
6.         packet->print();
7.     }
8.     if (VERIFICATION_OUTPUT)
9.     {
10.        packet->print(currentClockCycle, false);
11.    }
12.    if (!pimRank->isReservedRA(packet->row))
13.    {
14.        check(packet);
15.        updateState(packet);
16.    }
17.    sendToBank(packet);
18. }
```

Rank.cpp

```
1. void Rank::updateState(BusPacket* packet)
2. {
3.     auto addrMapping = config.addrMapping;
4.     if (packet->busPacketType == REF)
5.     {
6.         refreshWaiting = false;
7.         for (size_t i = 0; i < config.NUM_BANKS; i++)
8.         {
9.             bankStates[i].nextActivate = currentClockCycle + config.tRFC;
10.        }
11.    }
12.    else if (mode_ == dramMode::SB)
13.    {
14.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
15.        {
16.            updateBank(packet->busPacketType, bank, packet->row, bank == packet->bank,
17.                addrMapping.isSameBankgroup(bank, packet->bank));
18.        }
19.    }
20.    else
21.    {
22.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
23.        {
24.            updateBank(packet->busPacketType, bank, packet->row, (bank % 2) == packet->bank, true);
25.        }
26.    }
27. }
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

• measureCycle() → ... → **MemoryController::update()** → **Rank::receiveFromBus()** → **Rank::updateState()** → **Rank::updateBank()**

• **updateBank()**

2. Single Bank mode(= 일반적인 DRAM 동작) 검사 → 모든 Bank에 대해 updateBank를 호출하되, 내 명령의 직접적인 대상인지 정보를 넘김

- 대상 Bank: 명령에 따른 직접적인 latency(ex. tRP, tRC) 적용
- 같은 Bank Group: Bank Group 간 timing(ex. tRRDL, tCCDL) 적용
- 다른 Bank Group: 최소한의 간섭 timing(ex. tRRDS, tCCDS) 적용
- 다음 page에 계속

```
1. void Rank::updateState(BusPacket* packet)
2. {
3.     auto addrMapping = config.addrMapping;
4.     if (packet->busPacketType == REF)
5.     {
6.         refreshWaiting = false;
7.         for (size_t i = 0; i < config.NUM_BANKS; i++)
8.         {
9.             bankStates[i].nextActivate = currentClockCycle + config.tRFC;
10.        }
11.    }
12.    else if (mode_ == dramMode::SB)
13.    {
14.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
15.        {
16.            updateBank(packet->busPacketType, bank, packet->row, bank == packet->bank,
17.                       addrMapping.isSameBankgroup(bank, packet->bank));
18.        }
19.    }
20.    else
21.    {
22.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
23.        {
24.            updateBank(packet->busPacketType, bank, packet->row, (bank % 2) == packet->bank, true);
25.        }
26.    }
27. }
```

Rank.cpp

```
1. void Rank::updateBank(BusPacketType type, int bank, int row, bool targetBank, bool targetBankgroup)
2. {
3.     switch (type)
4.     {
5.         case READ:
6.             if (targetBank)
7.             {
8.                 bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
9.                                                         currentClockCycle + config.READ_TO_PRE_DELAY);
10.            }
11.            if (targetBankgroup)
12.            {
13.                bankStates[bank].nextRead =
14.                    max(bankStates[bank].nextRead,
15.                        currentClockCycle + max(config.tCCDL, config.BL / 2));
16.            }
17.            else
18.            {
19.                bankStates[bank].nextRead =
20.                    max(bankStates[bank].nextRead,
21.                        currentClockCycle + max(config.tCCDS, config.BL / 2));
22.            }
23.            bankStates[bank].nextWrite =
24.                max(bankStates[bank].nextWrite, currentClockCycle + config.READ_TO_WRITE_DELAY);
25.            break;
26.        case WRITE:
27.            // update state table
28.            if (targetBank)
29.            {
30.                bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
31.                                                         currentClockCycle + config.WRITE_TO_PRE_DELAY);
32.            }
33.            if (targetBankgroup)
34.            {
35.                bankStates[bank].nextRead =
36.                    max(bankStates[bank].nextRead,
37.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_LONG);
38.                bankStates[bank].nextWrite =
39.                    max(bankStates[bank].nextWrite,
40.                        currentClockCycle + max(config.BL / 2, config.tCCDL));
41.            }
42.            else
43.            {
44.                bankStates[bank].nextRead =
45.                    max(bankStates[bank].nextRead,
46.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_SHORT);
47.                bankStates[bank].nextWrite =
48.                    max(bankStates[bank].nextWrite,
49.                        currentClockCycle + max(config.BL / 2, config.tCCDS));
50.            }
51.            break;
52.        case ACTIVATE:
53.            if (targetBank)
54.            {
55.                bankStates[bank].currentBankState = RowActive;
56.                bankStates[bank].nextActivate = currentClockCycle + config.tRC;
57.                bankStates[bank].openRowAddress = row;
58.                bankStates[bank].nextWrite = currentClockCycle + (config.tRCdWR - config.AL);
59.                bankStates[bank].nextRead = currentClockCycle + (config.tRCdRD - config.AL);
60.                bankStates[bank].nextPrecharge = currentClockCycle + config.tRAS;
61.            }
62.            else
63.            {
64.                bankStates[bank].nextActivate =
65.                    (targetBankgroup
66.                     ? max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDL)
67.                     : max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDS));
68.                break;
69.        case PRECHARGE:
70.            if (targetBank)
71.            {
72.                bankStates[bank].currentBankState = Idle;
73.                bankStates[bank].nextActivate =
74.                    max(bankStates[bank].nextActivate, currentClockCycle + config.tRP);
75.            }
76.            break;
77.        case DATA:
78.            break;
79.        default:
80.            ERROR("Error - Unknown BusPacketType trying to be sent to Bank");
81.            exit(0);
82.            break;
83.    }
84. }
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

• measureCycle() → ... → **MemoryController::update()** → **Rank::receiveFromBus()** → **Rank::updateState()** → **Rank::updateBank()**

• **updateBank()**

3. HAB/PIM mode (= Multi Bank Mode) → 짝수 또는 홀수 번호의 Bank들을 동시에 Target Bank로 취급하여 업데이트

- 대상 Bank: 명령에 따른 직접적인 latency(ex. tRP, tRC) 적용
- 같은 Bank Group: Bank Group 간 timing(ex. tRRDL, tCCDL) 적용
- 다른 Bank Group: 최소한의 간섭 timing(ex. tRRDS, tCCDS) 적용

```
1. void Rank::updateState(BusPacket* packet)
2. {
3.     auto addrMapping = config.addrMapping;
4.     if (packet->busPacketType == REF)
5.     {
6.         refreshWaiting = false;
7.         for (size_t i = 0; i < config.NUM_BANKS; i++)
8.         {
9.             bankStates[i].nextActivate = currentClockCycle + config.tRFC;
10.        }
11.    }
12.    else if (mode_ == dramMode::SB)
13.    {
14.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
15.        {
16.            updateBank(packet->busPacketType, bank, packet->row, bank == packet->bank,
17.                       addrMapping.isSameBankgroup(bank, packet->bank));
18.        }
19.    }
20.    else
21.    {
22.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
23.        {
24.            updateBank(packet->busPacketType, bank, packet->row, (bank % 2) == packet->bank, true);
25.        }
26.    }
27. }
```

Rank.cpp

```
1. void Rank::updateBank(BusPacketType type, int bank, int row, bool targetBank, bool targetBankgroup)
2. {
3.     switch (type)
4.     {
5.         case READ:
6.             if (targetBank)
7.             {
8.                 bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
9.                                                         currentClockCycle + config.READ_TO_PRE_DELAY);
10.            }
11.            if (targetBankgroup)
12.            {
13.                bankStates[bank].nextRead =
14.                    max(bankStates[bank].nextRead,
15.                        currentClockCycle + max(config.tCCDL, config.BL / 2));
16.            }
17.            else
18.            {
19.                bankStates[bank].nextRead =
20.                    max(bankStates[bank].nextRead,
21.                        currentClockCycle + max(config.tCCDS, config.BL / 2));
22.            }
23.            bankStates[bank].nextWrite =
24.                max(bankStates[bank].nextWrite, currentClockCycle + config.READ_TO_WRITE_DELAY);
25.            break;
26.        case WRITE:
27.            // update state table
28.            if (targetBank)
29.            {
30.                bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
31.                                                         currentClockCycle + config.WRITE_TO_PRE_DELAY);
32.            }
33.            if (targetBankgroup)
34.            {
35.                bankStates[bank].nextRead =
36.                    max(bankStates[bank].nextRead,
37.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_LONG);
38.                bankStates[bank].nextWrite =
39.                    max(bankStates[bank].nextWrite,
40.                        currentClockCycle + max(config.BL / 2, config.tCCDL));
41.            }
42.            else
43.            {
44.                bankStates[bank].nextRead =
45.                    max(bankStates[bank].nextRead,
46.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_SHORT);
47.                bankStates[bank].nextWrite =
48.                    max(bankStates[bank].nextWrite,
49.                        currentClockCycle + max(config.BL / 2, config.tCCDS));
50.            }
51.            break;
52.        case ACTIVATE:
53.            if (targetBank)
54.            {
55.                bankStates[bank].currentBankState = RowActive;
56.                bankStates[bank].nextActivate = currentClockCycle + config.tRC;
57.                bankStates[bank].openRowAddress = row;
58.                bankStates[bank].nextWrite = currentClockCycle + (config.tRCdWR - config.AL);
59.                bankStates[bank].nextRead = currentClockCycle + (config.tRCdRD - config.AL);
60.                bankStates[bank].nextPrecharge = currentClockCycle + config.tRAS;
61.            }
62.            else
63.            {
64.                bankStates[bank].nextActivate =
65.                    (targetBankgroup
66.                     ? max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDL)
67.                     : max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDS));
68.                break;
69.            }
70.        case PRECHARGE:
71.            if (targetBank)
72.            {
73.                bankStates[bank].currentBankState = Idle;
74.                bankStates[bank].nextActivate =
75.                    max(bankStates[bank].nextActivate, currentClockCycle + config.tRP);
76.            }
77.            break;
78.        case DATA:
79.            break;
80.        default:
81.            ERROR("Error - Unknown BusPacketType trying to be sent to Bank");
82.            exit(0);
83.            break;
84.    }
85. }
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행
- 다음 page에서 sendToBank() 분석

```
1. void Rank::receiveFromBus(BusPacket* packet)
2. {
3.     if (DEBUG_BUS)
4.     {
5.         PRINTN(" -- R" << getChanId() << " Receiving On Bus   : ");
6.         packet->print();
7.     }
8.     if (VERIFICATION_OUTPUT)
9.     {
10.        packet->print(currentClockCycle, false);
11.    }
12.    if (!pimRank->isReservedRA(packet->row))
13.    {
14.        check(packet);
15.        updateState(packet);
16.    }
17.    sendToBank(packet);
18. }
```

Rank.cpp

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.            {
12.                pimRank->readHab(packet);
13.                packet->busPacketType = DATA;
14.                readReturnPacket.push_back(packet);
15.                readReturnCountdown.push_back(config.RL);
16.                break;
17.            }
18.        case WRITE:
19.            if (mode_ == dramMode::SB)
20.                writeSb(packet);
21.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
22.                pimRank->doPIM(packet);
23.            else
24.            {
25.                pimRank->writeHab(packet);
26.                delete (packet);
27.                break;
28.            }
29.        case ACTIVATE:
30.            if (DEBUG_CMD_TRACE)
31.            {
32.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
33.            }
34.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
35.                packet->column == 0x1f)
36.            {
37.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
38.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
39.                abmr2Even_ = (packet->bank == 0) ? true : abmr2Even_;
40.                abmr2Odd_ = (packet->bank == 1) ? true : abmr2Odd_;
41.            }
42.            if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
43.                (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
44.            {
45.                abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
46.                mode_ = dramMode::HAB;
47.                if (DEBUG_CMD_TRACE)
48.                {
49.                    PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
50.                }
51.            }
52.            delete (packet);
53.            break;
54.        case PRECHARGE:
55.            if (DEBUG_CMD_TRACE)
56.            {
57.                if (mode_ == dramMode::SB || packet->bank < 2)
58.                {
59.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
60.                }
61.                else
62.                {
63.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
64.                }
65.            }
66.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
67.            {
68.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
69.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
70.                if (sbmr1_ && sbmr2_)
71.                {
72.                    sbmr1_ = sbmr2_ = false;
73.                    mode_ = dramMode::SB;
74.                    if (DEBUG_CMD_TRACE)
75.                    {
76.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
77.                    }
78.                }
79.            }
80.            delete (packet);
81.            break;
82.        case REF:
83.            refreshWaiting = false;
84.            if (DEBUG_CMD_TRACE)
85.            {
86.                PRINTC(OUTLOG_CH_RA("REF"));
87.            }
88.            delete (packet);
89.            break;
90.        case DATA:
91.            delete (packet);
92.            break;
93.        default:
94.            ERROR("Error - Unknown BusPacketType trying to be sent to Bank");
95.            exit(0);
96.            break;
97.    }
98. }
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

hbm read test flow – measureCycle()

measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus() → sendToBank() → readSb()

• sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.1. Read → SB mode 시, readSb()(read Standard Bank) 호출

• PIM Register File(GRF_A, GRF_B)

• row address가 특정 address를 가리킬 때 수행됨

• PIMBlock의 general register file을 직접 읽음

• column address가 범위를 벗어나면 일반 memory 읽음

• 일반 DRAM Memory

• banks[]의 read() 함수 호출로 Memory 접근

• 발견된 이상한 점

• banks[]의 read() 함수 호출로 Memory 접근 시

• isReservedRA() == true 임에도 접근하고 있음 → ISSCC에서 허용 X

Index	Mode	CMD	RA13	Data Movement
1	Normal	WR	L	Data write to cell array
2		WR	H	Not available
3		RD	L	Data read to cell array
4		RD	H	Not available
5	FIM	WR	L	Data movement from PCU blick to cell array
6		WR	H	Data write to PCU register
7		RD	L	Data movement from cell array to PCU blick
8		RD	H	Not available

```

1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.            }
38.            if ((config.NUM_BANKS == 2 && abmr1Even_ && abmr1Odd_) ||
39.                (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.            {
41.                abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                mode = dramMode::HAB;
43.                if (DEBUG_CMD_TRACE)
44.                {
45.                    PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                }
47.            }
48.            delete (packet);
49.            break;
50.    }
51.    case PRECHARGE:
52.        if (DEBUG_CMD_TRACE)
53.        {
54.            void Rank::readSb(BusPacket* packet)
55.            {
56.                if (DEBUG_CMD_TRACE)
57.                {
58.                    if (packet->row == config.PIM_REG_RA)
59.                    {
60.                        if (0x08 <= packet->column && packet->column <= 0x0f)
61.                        {
62.                            PRINT(OUTLOG_GRF_A("READ_GRF_A"));
63.                        }
64.                        else if (0x18 <= packet->column && packet->column <= 0x1f)
65.                        {
66.                            PRINT(OUTLOG_GRF_B("READ_GRF_B"));
67.                        }
68.                    }
69.                    else if (pimRank->isReservedRA(packet->row))
70.                    {
71.                        PRINTC(GRAY, OUTLOG_ALL("READ"));
72.                    }
73.                    else
74.                    {
75.                        PRINT(OUTLOG_ALL("READ"));
76.                    }
77.                }
78.                #ifndef NO_STORAGE
79.                if (packet->row == config.PIM_REG_RA)
80.                {
81.                    if (0x08 <= packet->column && packet->column <= 0x0f)
82.                    {
83.                        *(packet->data) = pimRank->pimBlocks[packet->bank / 2].grfA[packet->column - 0x08];
84.                    }
85.                    else if (0x18 <= packet->column && packet->column <= 0x1f)
86.                    {
87.                        *(packet->data) = pimRank->pimBlocks[packet->bank / 2].grfB[packet->column - 0x18];
88.                    }
89.                    else
90.                    {
91.                        banks[packet->bank].read(packet);
92.                    }
93.                }
94.                else
95.                {
96.                    banks[packet->bank].read(packet);
97.                }
98.            }
99.            #endif
100.            break;
101.        }
102.    }
103.    void Bank::read(BusPacket* busPacket)
104.    {
105.        shared_ptr<DataStruct> rowHeadNode = rowEntries[busPacket->column];
106.        shared_ptr<DataStruct> foundNode = NULL;
107.        if ((foundNode = Bank::searchForRow(busPacket->row, rowHeadNode)) == NULL)
108.        {
109.            // found it
110.            // keep looking
111.            head = head->next;
112.            // if we get here, didn't find it
113.            return NULL;
114.        }
115.        *(busPacket->data) = foundNode->data;
116.    }

```

Rank.cpp

```

packet->bank = 0 → pimBlocks[0]
packet->bank = 1 → pimBlocks[0] (같은 블록)
packet->bank = 2 → pimBlocks[1]
packet->bank = 3 → pimBlocks[1] (같은 블록)
...

```

PIMBlock의 구성 예시

```

0x08~0x0f (8~15): GRF_A (General Register File A) - 8개
→ grfA[0] = column 0x08
→ grfA[1] = column 0x09
→ ...
→ grfA[7] = column 0x0f

0x18~0x1f (24~31): GRF_B (General Register File B) - 8개
→ grfB[0] = column 0x18
→ grfB[1] = column 0x19
→ ...
→ grfB[7] = column 0x1f

```

grfA, grfB의 column address

```

PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;

```

```

BurstType srf;
BurstType grfA[8];
BurstType grfB[8];

```

Configuration.h

PIMBlock.h

Bank.cpp

Bank.cpp

Samsung PIM Simulator hbm read test flow

hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()
- 2. Command Bus → receiveFromBus() → sendToBank() → isToggleCond()
- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별

- PIM mode 판별 → isToggleCond() 호출
 - pimOpMode_ : PIM mode 활성화
 - crfExit : EXIT command가 실행되었으면 1
 - toggleRa13h_ : host가 설정하는 제어 신호
 - 0: 일반 memory만 처리, reserved 영역 제외
 - 1: 모든 Bank의 모든 address를 처리
 - isReservedRA(packet->row)
 - packet의 address가 reserved 영역 접근하는지
 - toggleEvenBank & toggleOddBank
 - PIM 연산 시 example
 - toggleEvenBank_ = 1 일 때,
 - Bank 0, 2, 4 (짝수)는 PIM 연산 처리
 - Bank 1, 3, 5 (홀수)는 일반 DRAM 처리
 - toggleRa13h_ = 0 일 때, 해당 packet이 isReservedRA() = 1이면, PIM 연산 skip

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.     case READ:
6.         if (mode_ == dramMode::SB)
7.             readSb(packet);
8.         else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.             pimRank->doPIM(packet);
10.        else
11.            pimRank->readHab(packet);
12.        packet->busPacketType = DATA;
13.        readReturnPacket.push_back(packet);
14.        readReturnCountdown.push_back(config.RL);
15.        break;
16.     case WRITE:
17.         if (mode_ == dramMode::SB)
18.             writeSb(packet);
19.         else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.             pimRank->doPIM(packet);
21.         else
22.             pimRank->writeHab(packet);
23.         packet->busPacketType = DATA;
24.         readReturnPacket.push_back(packet);
25.         readReturnCountdown.push_back(config.RL);
26.         break;
27.     }
28.     delete (packet);
29.     break;
30. }
```

```
51. case PRECHARGE:
52.     if (DEBUG_CMD_TRACE)
53.     {
54.         if (mode_ == dramMode::SB || packet->bank < 2)
55.         {
56.             PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.         }
58.         else
59.         {
60.             PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.         }
62.     }
63.     if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.     {
65.         sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.         sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.         if (sbmr1_ && sbmr2_)
68.         {
69.             sbmr1_ = sbmr2_ = false;
70.             mode_ = dramMode::SB;
71.             if (DEBUG_CMD_TRACE)
72.             {
73.                 PRINTC(RED, OUTLOG_CH_RA("SB mode"));
74.             }
75.         }
76.     }
77. }
```

```
1. bool PIMRank::isToggleCond(BusPacket* packet)
2. {
3.     if (pimOpMode_ && !crfExit_)
4.     {
5.         if (toggleRa13h_)
6.         {
7.             if (toggleEvenBank_ && ((packet->bank & 1) == 0))
8.                 return true;
9.             else if (toggleOddBank_ && ((packet->bank & 1) == 1))
10.                return true;
11.            return false;
12.        }
13.        else if (toggleRa13h_ && !isReservedRA(packet->row))
14.        {
15.            if (toggleEvenBank_ && ((packet->bank & 1) == 0))
16.                return true;
17.            else if (toggleOddBank_ && ((packet->bank & 1) == 1))
18.                return true;
19.            return false;
20.        }
21.        return false;
22.    }
23.    return false;
24. }
```

```
1. unsigned inline isReservedRA(unsigned row)
2. {
3.     return (row & (1 << 13));
4. }
```

```
1. void PIMRank::controlPIM(BusPacket* packet)
2. {
3.     // ...
4.     pimOpMode_ = packet->data->u8Data[0] & 1; // Bit 0: PIM 모드 활성화
5.     toggleEvenBank_ = !(packet->data->u8Data[16] & 1); // Bit 16: 짝수 뱅크
6.     toggleOddBank_ = !(packet->data->u8Data[16] & 2); // Bit 17: 홀수 뱅크
7.     toggleRa13h_ = (packet->data->u8Data[16] & 4); // Bit 18: Row Bit 13 필터 모드
8. }
```

Samsung PIM Simulator hbm read test flow

hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()
- 2. Command Bus → receiveFromBus() → sendToBank() → doPIM()
 - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                ;
12.        }
13.    }
14.
15.    1. void PIMRank::doPIM(BusPacket* packet)
16.    2. {
17.    3.        PIMCmd cCmd;
18.    4.        packet->row = masked2accessibleRA(packet->row);
19.    5.        {
20.    6.            cCmd.fromInt(crf.data[pimPC_]);
21.    7.            if (DEBUG_CMD_TRACE)
22.    8.            {
23.    9.                PRINTC(CYAN, string((packet->busPacketType == READ) ? "READ ch" : "WRITE ch")
24.    10.                 << " ra" << getRankId() << " bg"
25.    11.                 << config.addMapping.bankGroupId(packet->bank) << " b" << packet->bank
26.    12.                 << " m" << packet->row << " c" << packet->column << " |" << " " << pimPC_
27.    13.                 << " " << cCmd.toString() << " @ " << currentClockCycle);
28.    14.            }
29.    15.            if (cCmd.type_ == PIMCmdType::EXIT)
30.    16.            {
31.    17.                crfExit_ = true;
32.    18.                break;
33.    19.            }
34.    20.            else if (cCmd.type_ == PIMCmdType::JUMP)
35.    21.            {
36.    22.                if (lastJumpIdx_ != pimPC_)
37.    23.                {
38.    24.                    if (cCmd.loopCounter_ > 0)
39.    25.                    {
40.    26.                        lastJumpIdx_ = pimPC_;
41.    27.                        numJumpToBeTaken_ = cCmd.loopCounter_;
42.    28.                    }
43.    29.                    if (numJumpToBeTaken_ > 0)
44.    30.                    {
45.    31.                        pimPC_ = cCmd.loopOffset_;
46.    32.                        numJumpToBeTaken_--;
47.    33.                    }
48.    34.                }
49.    35.            }
50.    36.        }
51.    37.    }
52.    38.    }
53.    39.    }
54.    40.    }
55.    41.    }
56.    42.    }
57.    43.    }
58.    44.    }
59.    45.    }
60.    46.    }
61.    47.    }
62.    48.    }
63.    49.    }
64.    50.    }
65.    51.    }
66.    52.    }
67.    53.    }
68.    54.    }
69.    55.    }
70.    56.    }
71.    57.    }
72.    58.    }
73.    59.    }
74.    60.    }
75.    61.    }
76.    62.    }
77.    63.    }
78.    64.    }
79.    65.    }
80.    66.    }
81.    67.    }
82.    68.    }
83.    69.    }
84.    70.    }
85.    71.    }
86.    72.    }
87.    73.    }
88.    74.    }
89.    75.    }
90.    76.    }
91.    77.    }
92.    78.    }
93.    79.    }
94.    80.    }
95.    81.    }
96.    82.    }
97.    83.    }
98.    84.    }
99.    85.    }
100.   86.    }
```

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.1. masked2accessibleRA(packet->row)

- 13번째 bit를 제외하고 LSB 12bit만 가져옴

1.2.2. pimPC_가 가리키는 command read 및 parsing

- EXIT, NOP, JUMP, FILL, MOV, MAD, ADD, MUL, MAC

1.2.3. command type별 PC값 처리

- EXIT : 종료 (crfExit_ = true)
- JUMP : PC를 loopOffset만큼 뒤로, loopCounter까지 반복
- FILL : 8번 반복 실행 (AUTO = 8번 고정됨)
- NOP : loopCounter만큼 stall

```
1. union crf_t
2. {
3.     uint32_t data[32];
4.     burstType bst[4];
5.     crf_t()
6.     {
7.         memset(data, 0, sizeof(uint32_t) * 32);
8.     }
9.     } crf;
10. }
11.
12. #if 메모리 레이아웃:
13.
14. crf.data[0] = 32 비트 명령어 #0
15.
16. crf.data[1] = 32 비트 명령어 #1
17.
18. crf.data[2] = 32 비트 명령어 #2
19.
20. ...
21.
22. crf.data[31] = 32 비트 명령어 #31
23.
24.
25. 최대 32개 명령어 저장 가능
```

```
1. void PIMCmd::fromInt(uint32_t val)
2. {
3.     type_ = PIMCmdType::fromBit(val, 4, 28));
4.     switch (type_)
5.     {
6.         case PIMCmdType::EXIT:
7.             break;
8.         case PIMCmdType::NOP:
9.             loopCounter_ = fromBit(val, 11, 0);
10.            break;
11.        case PIMCmdType::JUMP:
12.            loopCounter_ = fromBit(val, 17, 11);
13.            loopOffset_ = fromBit(val, 11, 0);
14.            break;
15.        case PIMCmdType::FILL:
16.            src2_ = PIMOpType::fromBit(val, 3, 25));
17.            dst_ = PIMOpType::fromBit(val, 3, 22));
18.            src0_ = PIMOpType::fromBit(val, 1, 12);
19.            dstIdx_ = fromBit(val, 4, 8);
20.            srcIdx_ = fromBit(val, 4, 4);
21.            src1Idx_ = fromBit(val, 4, 0);
22.            break;
23.        case PIMCmdType::MAD:
24.            src2_ = PIMOpType::fromBit(val, 3, 16);
25.            dst_ = PIMOpType::fromBit(val, 3, 22));
26.            src0_ = PIMOpType::fromBit(val, 1, 15);
27.            dstIdx_ = fromBit(val, 4, 8);
28.            srcIdx_ = fromBit(val, 4, 4);
29.            src1Idx_ = fromBit(val, 4, 0);
30.            break;
31.        case PIMCmdType::ADD:
32.            src2_ = PIMOpType::fromBit(val, 3, 25));
33.            dst_ = PIMOpType::fromBit(val, 3, 22));
34.            src0_ = PIMOpType::fromBit(val, 1, 15);
35.            dstIdx_ = fromBit(val, 4, 8);
36.            srcIdx_ = fromBit(val, 4, 4);
37.            src1Idx_ = fromBit(val, 4, 0);
38.            break;
39.        case PIMCmdType::MUL:
40.            src2_ = PIMOpType::fromBit(val, 3, 25));
41.            dst_ = PIMOpType::fromBit(val, 3, 22));
42.            src0_ = PIMOpType::fromBit(val, 1, 15);
43.            dstIdx_ = fromBit(val, 4, 8);
44.            srcIdx_ = fromBit(val, 4, 4);
45.            src1Idx_ = fromBit(val, 4, 0);
46.            break;
47.        default:
48.            break;
49.    }
50. }
```

(Instruction EXAMPLE)
CRF에 저장된 명령어:

```
CRF[0] = READ_BANK_A (bank 0 → GRF_A[0]) // 첫 번째 입력 읽기
CRF[1] = READ_BANK_B (bank 1 → GRF_A[1]) // 두 번째 입력 읽기
CRF[2] = AUTO_ADD (GRF_A[0], GRF_A[1] → GRF_B) // 덧셈 (8번 반복)
CRF[3] = WRITE_BANK (GRF_B → bank 0) // 결과 저장
CRF[4] = EXIT // 종료
```

실행 흐름:
pimPC=0: CRF[0] 실행 → 8번 반복 (8개 블록)
pimPC=1: CRF[1] 실행
pimPC=2: CRF[2] 실행 (FILL/AUTO이므로 8번 반복)
pimPC=3: CRF[3] 실행
pimPC=4: EXIT 도달 → crfExit=true

Samsung PIM Simulator hbm read test flow

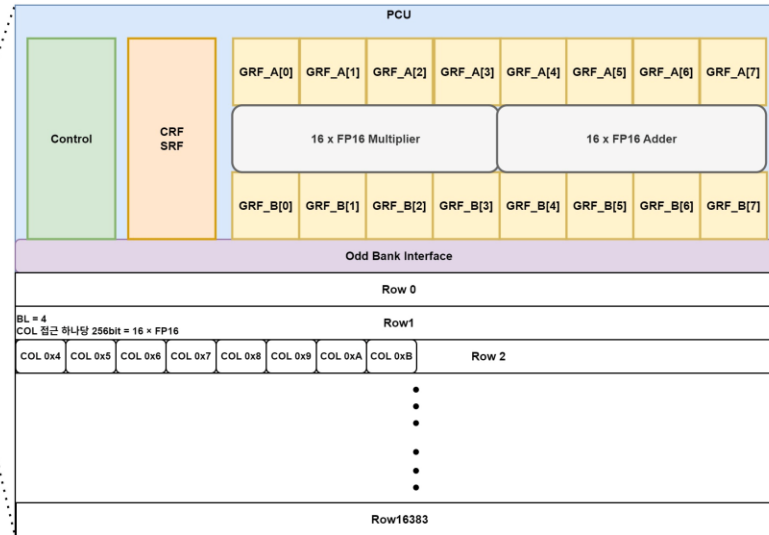
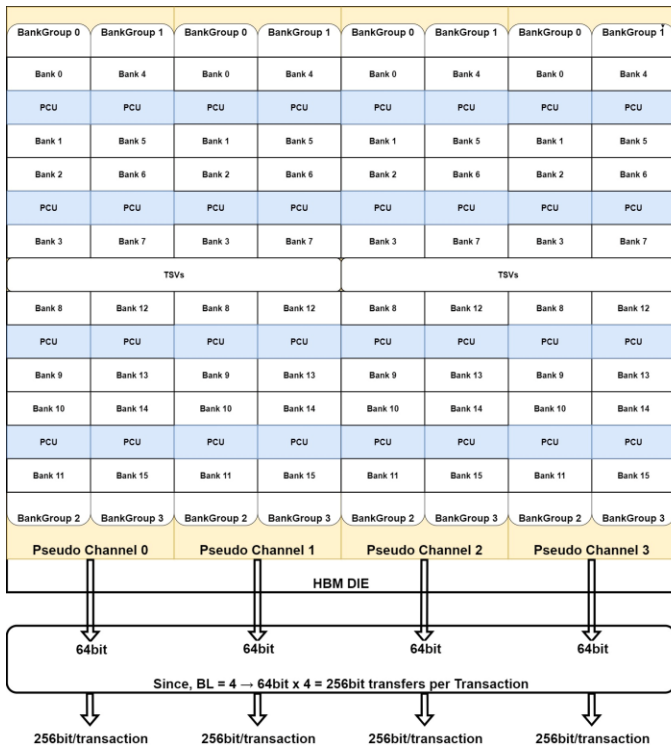
□ doPIMBlock() 분석 전 HBM2-PIM에 대한 분석

HBM2 spec

- NUM_BANK_GROUPS=4
- NUM_BANKS=16
- NUM_COLS=128
- NUM_ROWS=16384
- NUM_PIM_BLOCKS=8
- DEVICE_WIDTH=64
- BL=4
- RL=20
- WL=8

- Pseudo channel 당 256bit/transaction
- GRF 1개는 16 x FP16을 저장, GRF_A 8개, GRF_B 8개로 구성
- CRF(command register file), SRF(scalar register file)
- SIMD FP16 Multiplier & Adder

- 아래는 reference의 figure 5.에 제시된 MAC연산 example
- 이를 구체화한 그림
- simulator에서 이러한 operation이 doPIMBlock()에 구현되어 있음
- Operation들이 어떻게 control되는 지 분석 예정



- AAM mode에서 GRF_A/GRF_B의 index가 어떻게 정해지는지

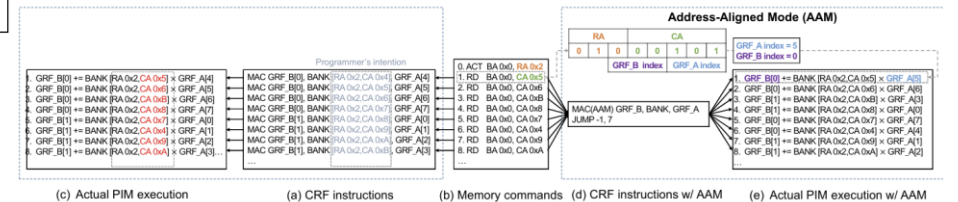
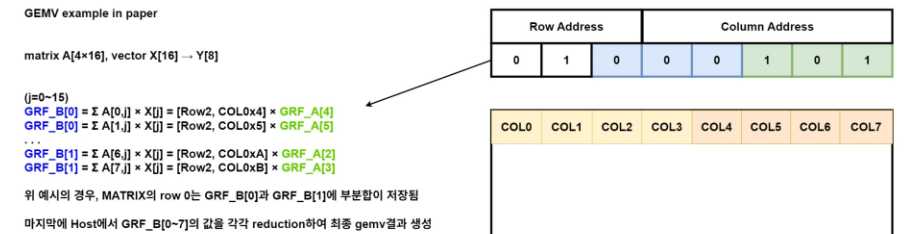


Fig. 5: Ordering MAC instructions in a GEMV microkernel. BA, RA and CA denote bank, row and column addresses.

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → ... → doPIM() → doPIMBlock()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- readOpd(pb, bst, type, packet, idx, is_auto, is_mac)

- PIM 명령 시 operand를 다양한 source에서 읽어옴
- PIMOpdType::A_OUT
- PIMOpdType::M_OUT

- 이전 계산 결과를 다음 연산의 입력으로 사용
- Direct Memory Access 없음

- PIMOpdType::EVEN_BANK

PIMOpdType::ODD_BANK

- 실제 DRAM Memory 읽기
- packet의 row/column 정보 사용하여 접근

- PIMOpdType::GRF_A → GRF에서 data 읽기

- is_auto = true: getGrfIdx()로 column addr 중 하위 3bit만 가져옴 (PPT page29 참조)
(ex. FILL시 같은 주소 8번 반복)
- is_auto = false: idx를 그대로 사용
(ex. MOV시 직접 GRF의 idx를 지정하여 사용)

PIMRank.cpp

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0, packet, cCmd.src0Idx, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst, packet, cCmd.dstIdx, is_auto, false);
15.    }
16.    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0, packet, cCmd.src0Idx, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1, packet, cCmd.src1Idx, cCmd.isAuto_, false);
23.
24.        if (cCmd.type_ == PIMCmdType::ADD)
25.            // dstBst = src0Bst + src1Bst;
26.            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
27.        else if (cCmd.type_ == PIMCmdType::MUL)
28.            // dstBst = src0Bst * src1Bst;
29.            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
30.
31.        writeOpd(pimblock_id, dstBst, cCmd.dst, packet, cCmd.dstIdx, cCmd.isAuto_, false);
32.    }
33.    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34.    {
35.        // ...
36.    }
37.}

1. void PIMRank::readOpd(int pb, BurstType& bst, PIMOpdType type, BusPacket* packet, int idx,
2.    bool is_auto, bool is_mac)
3. {
4.     idx = getGrfIdx(idx);
5.
6.     switch (type)
7.     {
8.         case PIMOpdType::A_OUT:
9.             bst = pimBlocks[pb].aOut;
10.            return;
11.         case PIMOpdType::M_OUT:
12.             bst = pimBlocks[pb].mOut;
13.            return;
14.         case PIMOpdType::EVEN_BANK:
15.             if (packet->bank % 2 != 0)
16.                 PRINT("Warning, CRF bank coding and bank id from packet are inconsistent");
17.             rank->banks[pb * 2].read(packet); // basically read from bank.
18.             bst = *(packet->data);
19.            return;
20.         case PIMOpdType::ODD_BANK:
21.             if (packet->bank % 2 == 0)
22.                 PRINT("Warning, CRF bank coding and bank id from packet are inconsistent");
23.             rank->banks[pb * 2 + 1].read(packet); // basically read from bank.
24.             bst = *(packet->data);
25.            return;
26.         case PIMOpdType::GRF_A:
27.             bst = pimBlocks[pb].grfA[(is_auto) ? getGrfIdx(packet->column) : idx];
28.            return;
29.         case PIMOpdType::GRF_B:
30.             if (is_auto)
31.                 bst = pimBlocks[pb].grfB[(is_mac) ? getGrfIdxHigh(packet->row, packet->column) : getGrfIdx(packet->column)];
32.             else
33.                 bst = pimBlocks[pb].grfB[idx];
34.            return;
35.         case PIMOpdType::SRF_M:
36.             bst.set(pimBlocks[pb].srf.fp16Data[idx]);
37.            return;
38.         case PIMOpdType::SRF_A:
39.             bst.set(pimBlocks[pb].srf.fp16Data[idx + 8]);
40.            return;
41.    }
42.}

Parameter들
• pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
• bst : 읽어온 data를 저장할 32byte burst
• type : data source 결정
• packet : READ/WRITE/CMD packet
• idx : data index (GRF index, SRF offset 등)
• is_auto : FILL등의 cmd로 자동 address 계산 여부
• is_mac : MAC 명령인지 여부

1. unsigned inline getGrfIdx(unsigned idx)
2. {
3.     return idx & 0x7;
4. }

1. unsigned inline getGrfIdxHigh(unsigned r, unsigned c)
2. {
3.     return ((r & 0x1) << 2 | ((c >> 3) & 0x3));
4. }
```


Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → ... → doPIM() → doPIMBlock()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- readOpd(pb, bst, type, packet, idx, is_auto, is_mac)

- PIM 명령 시 operand를 다양한 source에서 읽어옴
- PIMOpdType::GRF_B → GRF에서 data 읽기
 - is_auto = true & is_mac = true
 - getGrfIdxHigh()로 row address 하위 1bit, column address 상위 2bit 사용 (PPT page29 참조)
 - is_auto = true & is_mac = false
 - getGrfIdx()로 column address 하위 3bit만 가져옴
 - is_auto = false
 - idx를 그대로 사용
- PIMOpdType::SRF_M
 - srf에서 multiply에 사용할 FP16 data 가져옴
- PIMOpdType::SRF_A
 - srf에서 add에 사용할 FP16 data 가져옴

PIMRank.cpp

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type == PIMCmdType::FILL || cCmd.type == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0, packet, cCmd.src0Idx, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst, packet, cCmd.dstIdx, is_auto, false);
15.    }
16.    else if (cCmd.type == PIMCmdType::ADD || cCmd.type == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0, packet, cCmd.src0Idx, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1, packet, cCmd.src1Idx, cCmd.isAuto_, false);
23.
24.        if (cCmd.type == PIMCmdType::ADD)
25.        {
26.            // dstBst = src0Bst + src1Bst;
27.            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
28.        }
29.        else if (cCmd.type == PIMCmdType::MUL)
30.        {
31.            // dstBst = src0Bst * src1Bst;
32.            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
33.        }
34.        writeOpd(pimblock_id, dstBst, cCmd.dst, packet, cCmd.dstIdx, cCmd.isAuto_, false);
35.    }
36.}

33.     else if (cCmd.type == PIMCmdType::MAC || cCmd.type == PIMCmdType::MAD)
34.     {
35.         BurstType dstBst;
36.         BurstType src0Bst;
37.         BurstType src1Bst;
38.         readOpd(pimblock_id, src0Bst, cCmd.src0, packet, cCmd.src0Idx, cCmd.isAuto_, false);
39.         readOpd(pimblock_id, src1Bst, cCmd.src1, packet, cCmd.src1Idx, cCmd.isAuto_, false);
40.         readOpd(pimblock_id, dstBst, cCmd.dst, packet, cCmd.dstIdx, cCmd.isAuto_, false);
41.         if (cCmd.type == PIMCmdType::MAC)
42.             pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
43.         else if (cCmd.type == PIMCmdType::MAD)
44.             pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst);
45.         writeOpd(pimblock_id, dstBst, cCmd.dst, packet, cCmd.dstIdx, cCmd.isAuto_, false);
46.    }
47.}
```

Parameter들

- pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
- bst : 읽어온 data를 저장할 32byte burst
- type : data source 결정
- packet : READ/WRITE/CMD packet
- idx : data index (GRF index, SRF offset 등)
- is_auto : FILL등의 cmd로 자동 address 계산 여부
- is_mac : MAC 명령인지 여부

```
1. unsigned inline getGrfIdx(unsigned idx)
2. {
3.     return idx & 0x7;
4. }

1. unsigned inline getGrfIdxHigh(unsigned r, unsigned c)
2. {
3.     return ((r & 0x1) << 2 | ((c >> 3) & 0x3));
4. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- writeOpd(pb, bst, type, packet, idx, is_auto, is_mac)

- PIM 연산 후, 계산된 결과값을 dst Operand에 저장
- PIMOpdType::A_OUT
PIMOpdType::M_OUT
 - 각 PIM Block 내부에 있는 output buffer에 결과를 저장
- PIMOpdType::EVEN_BANK
PIMOpdType::ODD_BANK
 - 실제 DRAM Memory 쓰기
 - packet의 row/column 정보 사용하여 접근
- PIMOpdType::GRF_A → GRF에 결과 저장
 - is_auto = true: getGrfIdx()로 column addr 중 하위 3bit만 가져옴 (PPT page29 참조)
(ex. FILL시 같은 주소 8번 반복)
 - is_auto = false: idx를 그대로 사용
(ex. MOV시 직접 GRF의 idx를 지정하여 사용)

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
8.         if (cCmd.isRelu_)
9.         {
10.             for (int i = 0; i < 16; i++)
11.                 bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
12.             writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
13.         }
14.     }
15.     else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
16.     {
17.         BurstType dstBst;
18.         BurstType src0Bst;
19.         BurstType src1Bst;
20.         readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
21.         readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
22.         if (cCmd.type_ == PIMCmdType::ADD)
23.             // dstBst = src0Bst + src1Bst;
24.             pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
25.         else if (cCmd.type_ == PIMCmdType::MUL)
26.             // dstBst = src0Bst * src1Bst;
27.             pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
28.         writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
29.     }
30. }
31.
32.
33. else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34. {
35.     BurstType dstBst;
36.     void PIMRank::writeOpd(int pb, BurstType& bst, PIMOpdType type, BusPacket* packet, int idx,
37.         bool is_auto, bool is_mac)
38. {
39.     idx = getGrfIdx(idx);
40.     switch (type)
41.     {
42.         case PIMOpdType::A_OUT:
43.             pimBlocks[pb].aOut = bst;
44.             return;
45.         case PIMOpdType::M_OUT:
46.             pimBlocks[pb].mOut = bst;
47.             return;
48.         case PIMOpdType::EVEN_BANK:
49.             if (packet->bank % 2 != 0)
50.             {
51.                 PRINT("CRF bank coding and bank id from packet are inconsistent");
52.                 *(packet->data) = bst;
53.                 rank->banks[pb * 2].write(packet); // basically read from bank.
54.                 return;
55.             }
56.             case PIMOpdType::ODD_BANK:
57.                 if (packet->bank % 2 == 0)
58.                 {
59.                     PRINT("CRF bank coding and bank id from packet are inconsistent");
60.                     exit(-1);
61.                 }
62.                 *(packet->data) = bst;
63.                 rank->banks[pb * 2 + 1].write(packet); // basically read from bank.
64.                 return;
65.             case PIMOpdType::GRF_A:
66.                 pimBlocks[pb].grfA[(is_auto) ? getGrfIdx(packet->column) : idx] = bst;
67.                 return;
68.             case PIMOpdType::GRF_B:
69.                 if (is_auto)
70.                     pimBlocks[pb].grfB[(is_mac) ? getGrfIdxHigh(packet->row, packet->column)
71.                         : getGrfIdx(packet->column)] = bst;
72.                 else
73.                     pimBlocks[pb].grfB[idx] = bst;
74.                 return;
75.             case PIMOpdType::SRF_M:
76.                 pimBlocks[pb].srf = bst;
77.                 return;
78.             case PIMOpdType::SRF_A:
79.                 pimBlocks[pb].srf = bst;
80.                 return;
81.         }
82.     }
83. }
```

PIMRank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- writeOpd(pb, bst, type, packet, idx, is_auto, is_mac)

- PIM 연산 후, 계산된 결과값을 dst Operand에 저장
- PIMOpdType::GRF_B → GRF에 결과 저장

- is_auto = true & is_mac = true

- getGrfldxHigh()로 row address 하위 1bit, column address상위 2bit 사용 (PPT page29 참조)

- is_auto = true & is_mac = false

- getGrfldx()로 column addr중 하위 3bit만 가져옴

- is_auto = false

- idx를 그대로 사용

- PIMOpdType::SRF_M

- PIMOpdType::SRF_M

- Scalar Register File에 값을 update

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
8.         if (cCmd.isRelu_)
9.         {
10.             for (int i = 0; i < 16; i++)
11.                 bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
12.             writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
13.         }
14.     }
15.     else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
16.     {
17.         BurstType dstBst;
18.         BurstType src0Bst;
19.         BurstType src1Bst;
20.         readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
21.         readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
22.         if (cCmd.type_ == PIMCmdType::ADD)
23.             // dstBst = src0Bst + src1Bst;
24.             pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
25.         else if (cCmd.type_ == PIMCmdType::MUL)
26.             // dstBst = src0Bst * src1Bst;
27.             pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
28.         writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
29.     }
30. }
31.
32.
33. else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34. {
35.     BurstType dstBst;
36.     BurstType src0Bst;
37.     BurstType src1Bst;
38.     readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
39.     readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
40.     if (cCmd.type_ == PIMCmdType::MAC)
41.         // dstBst = src0Bst * src1Bst;
42.         pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
43.     else if (cCmd.type_ == PIMCmdType::MAD)
44.         // dstBst = src0Bst + src1Bst;
45.         pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
46.     writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
47. }
48. }
```

Parameter들

- pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
- bst : write할 32byte burst data
- type : data source 결정
- packet : READ/WRITE/CMD packet
- idx : data dst index (GRF index)
- is_auto : FILL등의 cmd로 자동 address 계산 여부
- is_mac : MAC 명령인지 여부

PIMRank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- add(dstBst, src0Bst, src1Bst)

- 개별 PIM Block이 수행하는 Vector 덧셈 연산 수행
- Type별 덧셈 수행
 - FP16
 - 16개의 element를 각각 더함
 - FP32
 - 8개의 element를 각각 더함
 - 그 외
 - BurstType operator + 호출

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
15.    }
16.    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
23.
24.        if (cCmd.type_ == PIMCmdType::ADD)
25.        {
26.            // dstBst = src0Bst + src1Bst;
27.            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
28.        }
29.        else if (cCmd.type_ == PIMCmdType::MUL)
30.        {
31.            // dstBst = src0Bst * src1Bst;
32.            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
33.        }
34.        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
35.    }
36. }

33. else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34. {
35.     BurstType dstBst;
36.     BurstType src0Bst;
37.     BurstType src1Bst;
38.     bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;
39.
40.     readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
41.     readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
42.     if (is_mac)
43.     {
44.         readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
45.         // dstBst = src0Bst * src1Bst + dstBst;
46.         pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
47.     }
48.     else
49.     {
50.         BurstType src2Bst;
51.         readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src2Idx_, cCmd.isAuto_, is_mac);
52.         // dstBst = src0Bst * src1Bst + src2Bst;
53.         pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
54.     }
55.     writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
56. }
57. }

1. void PIMBlock::add(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst)
2. {
3.     if (pimPrecision_ == FP16)
4.     {
5.         for (int i = 0; i < 16; i++)
6.         {
7.             dstBst.fp16Data[i] = src0Bst.fp16Data[i] + src1Bst.fp16Data[i];
8.         }
9.     }
10.    else if (pimPrecision_ == FP32)
11.    {
12.        for (int i = 0; i < 8; i++)
13.        {
14.            dstBst.fp32Data[i] = src0Bst.fp32Data[i] + src1Bst.fp32Data[i];
15.        }
16.    }
17.    else
18.    {
19.        dstBst = src0Bst + src1Bst;
20.    }
21. }

1. BurstType operator+(const BurstType& rhs) const
2. {
3.     BurstType ret;
4.     for (int i = 0; i < 16; i++)
5.     {
6.         ret.fp16Data[i] = fp16Data[i] + rhs.fp16Data[i];
7.     }
8.     return ret;
9. }
```

PIMRank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- mul(dstBst, src0Bst, src1Bst)

- 개별 PIM Block이 수행하는 vector 곱셈 연산 수행
- Type별 곱셈 수행
 - FP16
 - 16개의 element를 각각 곱함
 - FP32
 - 8개의 element를 각각 곱함
 - 그 외
 - BurstType operator * 호출

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
15.    }
16.    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
23.
24.        if (cCmd.type_ == PIMCmdType::ADD)
25.        {
26.            // dstBst = src0Bst + src1Bst;
27.            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
28.        }
29.        else if (cCmd.type_ == PIMCmdType::MUL)
30.        {
31.            // dstBst = src0Bst * src1Bst;
32.            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
33.        }
34.        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
35.    }
36. }
37.
38. else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
39. {
40.     BurstType dstBst;
41.     BurstType src0Bst;
42.     BurstType src1Bst;
43.     bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;
44.
45.     readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
46.     readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
47.     if (is_mac)
48.     {
49.         readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
50.         // dstBst = src0Bst * src1Bst + dstBst;
51.         pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
52.     }
53.     else
54.     {
55.         BurstType src2Bst;
56.         readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src2Idx_, cCmd.isAuto_, is_mac);
57.         // dstBst = src0Bst * src1Bst + src2Bst;
58.         pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
59.     }
60.     writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
61. }
62.
63. 1. void PIMBlock::mul(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst)
64. 2. {
65. 3.     if (pimPrecision_ == FP16)
66. 4.     {
67. 5.         for (int i = 0; i < 16; i++)
68. 6.         {
69. 7.             dstBst.fp16Data[i] = src0Bst.fp16Data[i] * src1Bst.fp16Data[i];
80. 8.         }
91. 9.     }
10.    else if (pimPrecision_ == FP32)
11.    {
12.    13.        for (int i = 0; i < 8; i++)
14.    14.        {
15.    15.            dstBst.fp32Data[i] = src0Bst.fp32Data[i] * src1Bst.fp32Data[i];
16.    16.        }
17.    17.    }
18.    else
19.    {
20.        dstBst = src0Bst * src1Bst;
21.    }
22. }
```

PIMRank.cpp

```
1. BurstType operator*(const BurstType& rhs) const
2. {
3.     BurstType ret;
4.     for (int i = 0; i < 16; i++)
5.     {
6.         ret.fp16Data[i] = fp16Data[i] * rhs.fp16Data[i];
7.     }
8.     return ret;
9. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- mac(dstBst, src0Bst, src1Bst)

- 개별 PIM Block이 수행하는 vector MAC 연산 수행

- Type별 곱셈 수행

- FP16

- 16개의 element를 각각 MAC

- FP32

- 8개의 element를 각각 MAC

- 그 외

- BurstType operator * 와 + 호출

- mad → $c = a * b + c$

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
8.         if (cCmd.isRelu_)
9.         {
10.             for (int i = 0; i < 16; i++)
11.                 bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
12.             writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
13.         }
14.     }
15.     else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
16.     {
17.         BurstType dstBst;
18.         BurstType src0Bst;
19.         BurstType src1Bst;
20.         readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
21.         readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
22.         if (cCmd.type_ == PIMCmdType::ADD)
23.             pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
24.         else if (cCmd.type_ == PIMCmdType::MUL)
25.             pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst);
26.         writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
27.     }
28. }

33. else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34. {
35.     BurstType dstBst;
36.     BurstType src0Bst;
37.     BurstType src1Bst;
38.     bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;
39.     readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
40.     readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
41.     if (is_mac)
42.     {
43.         readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
44.         // dstBst = src0Bst * src1Bst + dstBst;
45.         pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
46.     }
47.     else
48.     {
49.         BurstType src2Bst;
50.         readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src2Idx_, cCmd.isAuto_, is_mac);
51.         // dstBst = src0Bst * src1Bst + src2Bst;
52.         pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
53.     }
54.     writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
55. }

1. void PIMBlock::mac(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst)
2. {
3.     if (pimPrecision_ == FP16)
4.     {
5.         for (int i = 0; i < 16; i++)
6.         {
7.             dstBst.fp16Data[i] = src0Bst.fp16Data[i] * src1Bst.fp16Data[i] + dstBst.fp16Data[i];
8.         }
9.         DEBUG("MAC " << src0Bst.hexToStr2() << "*" << src1Bst.hexToStr2() << " "
10.             << dstBst.hexToStr2());
11.     }
12.     else if (pimPrecision_ == FP32)
13.     {
14.         for (int i = 0; i < 8; i++)
15.         {
16.             dstBst.fp32Data[i] = src0Bst.fp32Data[i] * src1Bst.fp32Data[i] + dstBst.fp32Data[i];
17.         }
18.     }
19.     else
20.     {
21.         dstBst = src0Bst * src1Bst + dstBst;
22.     }
23. }
```


Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- mad(dstBst, src0Bst, src1Bst, src2Bst)

- 개별 PIM Block이 수행하는 vector MAD 연산 수행

- Type별 곱셈 수행

- FP16

- 16개의 element를 각각 MAD

- FP32

- 8개의 element를 각각 MAD

- 그 외

- BurstType operator * 와 + 호출

- mad → $d = a * b + c$

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
8.         if (cCmd.isRelu_)
9.         {
10.             for (int i = 0; i < 16; i++)
11.                 bst.u16Data[i] = (bst.u16Data[i] & (1 << 15)) ? 0 : bst.u16Data[i];
12.             writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
13.         }
14.     }
15.     else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
16.     {
17.         BurstType dstBst;
18.         BurstType src0Bst;
19.         BurstType src1Bst;
20.         readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
21.         readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
22.         if (cCmd.type_ == PIMCmdType::ADD)
23.         {
24.             if (cCmd.type_ == PIMCmdType::ADD)
25.             {
26.                 1. void PIMBlock::mad(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst, BurstType& src2Bst)
27.                 2. {
28.                 3.                     if (pimPrecision_ == FP16)
29.                 4.                     {
30.                 5.                         for (int i = 0; i < 16; i++)
31.                 6.                         {
32.                             dstBst.fp16Data[i] =
33.                                 src0Bst.fp16Data[i] * src1Bst.fp16Data[i] + src2Bst.fp16Data[i];
34.                         }
35.                     }
36.                     else if (pimPrecision_ == FP32)
37.                     {
38.                         for (int i = 0; i < 8; i++)
39.                         {
40.                             dstBst.fp32Data[i] =
41.                                 src0Bst.fp32Data[i] * src1Bst.fp32Data[i] + src2Bst.fp32Data[i];
42.                         }
43.                     }
44.                     else
45.                         dstBst = src0Bst * src1Bst + src2Bst;
46.                 }
47.             }
48.             else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
49.             {
50.                 BurstType dstBst;
51.                 BurstType src0Bst;
52.                 BurstType src1Bst;
53.                 bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;
54.                 readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
55.                 readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
56.                 if (is_mac)
57.                 {
58.                     readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
59.                     // dstBst = src0Bst * src1Bst + dstBst;
60.                     pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
61.                 }
62.                 else
63.                 {
64.                     BurstType src2Bst;
65.                     readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src2Idx_, cCmd.isAuto_, is_mac);
66.                     // dstBst = src0Bst * src1Bst + src2Bst;
67.                     pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
68.                 }
69.                 writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
70.             }
71.         }
72.         else if (cCmd.type_ == PIMCmdType::NOP && packet->busPacketType == WRITE)
73.         {
74.             = getGrfIdx(packet->column);
75.             ->bank = 0)
76.             et->data) = pimBlocks[pimblock_id].grfA[grf_id];
77.             banks[pimblock_id * 2].write(packet); // basically read from bank;
78.             packet->bank = 1)
79.             et->data) = pimBlocks[pimblock_id].grfB[grf_id];
80.             banks[pimblock_id * 2 + 1].write(packet); // basically read from bank.
81.         }
82.     }
83. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- cCmd.type_ == PIMCmdType::NOP && packet->busPacketType == WRITE
- PIM Mode에서 Packet이 Write 동작을 요청했으며, 내부에서 연산을 수행하지 않을 때(=NOP), Register Data를 Memory로 Write Back
- Bank == 0 (= even Bank)인 경우
 - pimBlocks[pimblock_id].grfA[grf_id]에서 data를 꺼내 packet->data에 copy
 - packet->data를 bank에 write
- Bank == 1 (= odd Bank)인 경우
 - pimBlocks[pimblock_id].grfB[grf_id]에서 data를 꺼내 packet->data에 copy
 - packet->data를 bank에 write

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.         1. void Bank::write(const BusPacket* busPacket)
8.         2. {
9.             // TODO: move all the error checking to BusPacket so once we have a bus
10.            // packet,
11.            // we know the fields are all legal
12.            6.
13.            if (busPacket->column >= numCols)
14.            {
15.                ERROR("Error - Bus Packet column " << busPacket->column << " out of bounds");
16.                exit(-1);
17.            }
18.            // head of the list we need to search
19.            shared_ptr<DataStruct> rowHeadNode = rowEntries[busPacket->column];
20.            shared_ptr<DataStruct> foundNode = NULL;
21.            if ((foundNode = Bank::searchForRow(busPacket->row, rowHeadNode)) == NULL)
22.            {
23.                // not found
24.                shared_ptr<DataStruct> newRowNode = make_shared<DataStruct>();
25.                // DataStruct* newRowNode = (DataStruct*)malloc(sizeof(DataStruct));
26.                21.
27.                // insert at the head for speed
28.                // TODO: Optimize this data structure for speedier lookups?
29.                newRowNode->row = busPacket->row;
30.                if (busPacket->data)
31.                {
32.                    newRowNode->data = *(busPacket->data);
33.                    newRowNode->next = rowHeadNode;
34.                    rowEntries[busPacket->column] = newRowNode;
35.                }
36.            }
37.            else
38.            {
39.                // found it, just plaster in the new data
40.                foundNode->data = *(busPacket->data);
41.                if (DEBUG_BANKS)
42.                {
43.                    PRINTN(" -- Bank " << busPacket->bank << " writing to physical address 0x" << hex
44.                        << busPacket->physicalAddress << dec << ":");
45.                    busPacket->printData();
46.                    PRINT("");
47.                }
48.            }
49.        }
50.    }
51.    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
52.    {
53.        BurstType dstBst;
54.        BurstType src0Bst;
55.        BurstType src1Bst;
56.        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;
57.        39.
58.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
59.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
60.        if (is_mac)
61.        {
62.            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
63.            // dstBst = src0Bst * src1Bst + dstBst;
64.            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
65.        }
66.        else
67.        {
68.            BurstType src2Bst;
69.            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src2Idx_, cCmd.isAuto_, is_mac);
70.            // dstBst = src0Bst * src1Bst + src2Bst;
71.            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
72.        }
73.        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
74.    }
75.    else if (cCmd.type_ == PIMCmdType::NOP && packet->busPacketType == WRITE)
76.    {
77.        int grf_id = getGrfIdx(packet->column);
78.        if (packet->bank == 0)
79.        {
80.            *(packet->data) = pimBlocks[pimblock_id].grfA[grf_id];
81.            rank->banks[pimblock_id * 2].write(packet); // basically read from bank;
82.        }
83.        else if (packet->bank == 1)
84.        {
85.            *(packet->data) = pimBlocks[pimblock_id].grfB[grf_id];
86.            rank->banks[pimblock_id * 2 + 1].write(packet); // basically read from bank.
87.        }
88.    }
89. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.3. Read → PIM mode 인지 판별 → 아닐 시 → readHab() 수행

readHab()

- READ 명령 시, PIM 연산을 위해 Bank data를 Reg로 Load
- isReservedRA() == true이면 skip
- isReservedRA() == false 시
 - Read한 Data를 PIM Reg(GRF_B)에 넣음
 - 모든 PIM Blocks에 대해 수행 → Even/Odd Banks 중 선택
 - Even/Odd Banks 는 packet->bank를 통해 선택 (ex. Packet->bank == 1이면 Odd Banks)

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE")) << "
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_PACKET_COLUMN == 0x1f)
31.            {
32.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even;
33.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd;
34.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even;
35.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd;
36.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_
37.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_
38.                    {
39.                        abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
40.                        mode_ = dramMode::HAB;
41.                        if (DEBUG_CMD_TRACE)
42.                        {
43.                            PRINTC(RED, OUTLOG_CH_RA("HAB")) << " tag : " << packet->tag;
44.                        }
45.                    }
46.                }
47.            }
48.            delete (packet);
49.            break;
50.    }
51.    case PRECHARGE:
52.        if (DEBUG_CMD_TRACE)
53.        {
54.            if (mode_ == dramMode::SB || packet->bank < 2)
55.            {
56.                PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.            }
58.            else
59.            {
60.                PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.            }
62.        }
63.        if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.        {
65.            sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.            sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.            if (sbmr1_ && sbmr2_)
68.            {
69.                in2_ = false;
70.                Mode::SB;
71.                ID_TRACE
72.                ED, OUTLOG_CH_RA("SB mode"));
73.            }
74.        }
75.    }
76.    case DATA:
77.        delete (packet);
78.        break;
79.    default:
80.        ERROR("Error - Unknown BusPacketType trying to be sent to Bank");
81.        exit(0);
82.        break;
83.    }
84.}
```

```
1. void PIMRank::readHab(BusPacket* packet)
2. {
3.     if (isReservedRA(packet->row)) // ignored
4.     {
5.         PRINTC(GRAY, OUTLOG_ALL("READ"));
6.     }
7.     else
8.     {
9.         PRINTC(GRAY, OUTLOG_ALL("BANK_TO_PIM"));
10.        #ifndef NO_STORAGE
11.        {
12.            int grf_id = getGrfIdx(packet->column);
13.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
14.            {
15.                rank->banks[pb * 2 + packet->bank].read(packet);
16.                pimBlocks[pb].grfB[grf_id] = *(packet->data);
17.            }
18.        }
19.        #endif
20.    }
21.}
```

```
1. unsigned inline getGrfIdx(unsigned idx)
2. {
3.     return idx & 0x7;
4. }
```

PIMRank.cpp

Rank.cpp

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.            {
65.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.
68.                if (sbmr1_ && sbmr2_)
69.                {
70.                    sbmr1_ = sbmr2_ = false;
71.                    mode_ = dramMode::SB;
72.                    if (DEBUG_CMD_TRACE)
73.                    {
74.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
75.                    }
76.                }
77.            }
78.            delete (packet);
79.            break;
80.        case REF:
81.            refreshWaiting = false;
82.            if (DEBUG_CMD_TRACE)
83.            {
84.                PRINT(OUTLOG_CH_RA("REF"));
85.            }
86.            delete (packet);
87.            break;
88.        case DATA:
89.            delete (packet);
90.            break;
91.        default:
92.            ERROR("Error - Unknown BusPacketType trying to be sent to Bank");
93.            exit(0);
94.            break;
95.    }
96. }
97.
98. Rank.cpp
99.
100.
101. }
```

command type별 처리 진행

1.4. Read → read Data & return 할 packet 저장 & Read Latency 설정

1.4.1. packet->busPacketType = DATA

- READ command packet을 DATA packet으로 변환

1.4.2. readReturnPacket.push_back(packet)

- Data를 RL(Read Latency)만큼 기다렸다가 Data Bus로 내보낼 때까지 임시로 buffer에 저장

1.4.3. readReturnCountdown.push_back(config.RL)

- DATA packet이 bus로 나가기까지 기다려야 할 시간 (= Read Latency)을 설정

Samsung PIM Simulator hbm read test flow

hbm read test flow – measureCycle()

measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case WRITE

```

1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                1. void Rank::writeSb(BusPacket* packet)
29.                2. {
30.                3.             if (DEBUG_CMD_TRACE)
31.                4.             {
32.                5.                 if (packet->row == config.PIM_REG_RA || pimRank->isReservedRA(packet->row))
33.                6.                 {
34.                7.                     PRINTC(GRAY, OUTLOG_ALL("WRITE"));
35.                8.                 }
36.                9.                 else
37.                10.                {
38.                11.                    PRINT(OUTLOG_ALL("WRITE"));
39.                12.                }
40.                13.            }
41.            }
42.            14. #ifndef NO_STORAGE
43.            15.             if (!!(packet->row == config.PIM_REG_RA) && !pimRank->isReservedRA(packet->row))
44.            16.             {
45.            17.                 banks[packet->bank].write(packet);
46.            18.             }
47.            19.             #endif
48.            delete (packet);
49.            break;
50.        }
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.            }
59.            void Bank::write(const BusPacket* busPacket)
60.            1. {
61.            2.             // TODO: move all the error checking to BusPacket so once we have a bus
62.            3.             // packet,
63.            4.             // we know the fields are all legal
64.            5.             if (busPacket->column >= numCols)
65.            6.             {
66.            7.                 ERROR("Error - Bus Packet column " << busPacket->column << " out of bounds");
67.            8.                 exit(-1);
68.            9.             }
69.            10.            // head of the list we need to search
69.            11.            shared_ptr<DataStruct> rowHeadNode = rowEntries[busPacket->column];
70.            12.            shared_ptr<DataStruct> foundNode = NULL;
71.            13.            if ((foundNode = Bank::searchForRow(busPacket->row, rowHeadNode)) == NULL)
72.            14.            {
73.            15.                // not found
74.            16.                shared_ptr<DataStruct> newRowNode = make_shared<DataStruct>();
75.            17.                // DataStruct* newRowNode = (DataStruct*)malloc(sizeof(DataStruct));
76.            18.                // insert at the head for speed
77.            19.                // TODO: Optimize this data structure for speedier lookups?
78.            20.                newRowNode->row = busPacket->row;
79.            21.                if (busPacket->data)
80.            22.                {
81.            23.                    newRowNode->data = *(busPacket->data);
82.            24.                    newRowNode->next = rowHeadNode;
83.            25.                    rowEntries[busPacket->column] = newRowNode;
84.            26.                }
85.            27.            }
86.            28.            else
87.            29.            {
88.            30.                // found it, just plaster in the new data
89.            31.                foundNode->data = *(busPacket->data);
90.            32.                if (DEBUG_BANKS)
91.            33.                {
92.            34.                    PRINTN(" -- Bank " << busPacket->bank << " writing to physical address 0x" << hex
93.            35.                    << busPacket->physicalAddress << dec << ":");
94.            36.                    busPacket->printData();
95.            37.                    PRINT("");
96.            38.                }
97.            39.            }
98.            40.            }
99.            41.        }
100.        42.    }
101.    }

```

Bank.cpp

<Data movement according to operation mode>				
Index	Mode	CMD	RA13	Data Movement
1	Normal	WR	L	Data write to cell array
2		WR	H	Not available
3		RD	L	Data read to cell array
4		RD	H	Not available
5	FIM	WR	L	Data movement from PCU bick to cell array
6		WR	H	Data write to PCU register
7		RD	L	Data movement from cell array to PCU bick
8		RD	H	Not available

command type별 처리 진행

2.1. Write → SB mode 시, writeSb() (write Standard Bank) 호출

- Write를 수행하지 않는 경우
 - PIM register 접근을 위한 특정 row address(=PIM_REG_RA)
 - isResrvedRA() == true인 경우
- 일반 DRAM Memory
 - banks[]의 write() 함수 호출로 Memory 접근

PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMRA = 0x2fff;

Configuration.h

BurstType srf;
BurstType grfA[8];
BurstType grfB[8];

PIMBlock.h

```

1. unsigned inline isReservedRA(unsigned row)
2. {
3.     return (row & (1 << 13));
4. }

```

PIMRank.h

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case WRITE

command type별 처리 진행

2.2. Write → PIM mode 인지 판별 → doPIM() 수행

- PPT page 30 ~ 39와 동일 동작 수행

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.                if ((config.NUM_BANKS == 2 && abmr1Even_ && abmr1Odd_) ||
38.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ &&
39.                     abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.            break;
64.    }
65.}

1. void PIMRank::doPIM(BusPacket* packet)
2. {
3.     PIMCmd cCmd;
4.     packet->row = masked2accessibleRA(packet->row);
5.     do
6.     {
7.         cCmd.fromInt(crf.data[pimPC_]);
8.         if (DEBUG_CMD_TRACE)
9.         {
10.            PRINTC(CYAN, string((packet->busPacketType == READ) ? "READ ch" : "WRITE ch")
11.                << " rankId: " << getRankId() << " bank: " << packet->bank << " column: " << packet->column << " row: " << packet->row << " loopCounter: " << cCmd.loopCounter);
12.        }
13.        if (cCmd.type == PIMCmdType::EXIT)
14.        {
15.            crfExit_ = true;
16.            break;
17.        }
18.        else if (cCmd.type == PIMCmdType::JUMP)
19.        {
20.            if (lastJumpIdx_ != pimPC_)
21.            {
22.                if (cCmd.loopCounter > 0)
23.                {
24.                    lastJumpIdx_ = pimPC_;
25.                    numJumpToBeTaken_ = cCmd.loopCounter;
26.                }
27.                if (numJumpToBeTaken_ > 0)
28.                {
29.                    pimPC_ = cCmd.loopOffset;
30.                    numJumpToBeTaken_--;
31.                }
32.            }
33.        }
34.    } while (cCmd.type == PIMCmdType::JUMP);
35.}

1. unsigned inline masked2accessibleRA(unsigned row)
2. {
3.     return (row & ((1 << 13) - 1));
4. }

38. else
39. {
40.     if (cCmd.type == PIMCmdType::FILL || cCmd.isAuto_)
41.     {
42.         if (lastRepeatIdx_ != pimPC_)
43.         {
44.             lastRepeatIdx_ = pimPC_;
45.             numRepeatToBeDone_ = 8 - 1;
46.         }
47.         if (numRepeatToBeDone_ > 0)
48.         {
49.             pimPC_--;
50.             numRepeatToBeDone_--;
51.         }
52.         else
53.             lastRepeatIdx_ = -1;
54.     }
55.     else if (cCmd.type == PIMCmdType::NOP)
56.     {
57.         if (lastRepeatIdx_ != pimPC_)
58.         {
59.             lastRepeatIdx_ = pimPC_;
60.             numRepeatToBeDone_ = cCmd.loopCounter;
61.         }
62.         if (numRepeatToBeDone_ > 0)
63.         {
64.             pimPC_--;
65.             numRepeatToBeDone_--;
66.         }
67.         else
68.             lastRepeatIdx_ = -1;
69.     }
70.     for (int pimblock_id = 0; pimblock_id < config.NUM_PIM_BLOCKS; pimblock_id++)
71.     {
72.         doPIMBlock(packet, cCmd, pimblock_id);
73.         if (DEBUG_PIM_BLOCK && pimblock_id == 0)
74.         {
75.             PRINTC(BANK_R" " << packet->data->fp16ToStr());
76.             PRINTC("[CMD]" << bitset<32>(cCmd.toInt()) << " (" << cCmd.toString() << ")");
77.             PRINTC(pimBlocks[pimblock_id].print());
78.             PRINTC("-----");
79.         }
80.     }
81.     pimPC_++;
82.     // EXIT check
83.     PIMCmd next_cmd;
84.     next_cmd.fromInt(crf.data[pimPC_]);
85.     if (next_cmd.type == PIMCmdType::EXIT)
86.     {
87.         crfExit_ = true;
88.     } while (cCmd.type == PIMCmdType::JUMP);
89. }

PIMRank.cpp
```

Samsung PIM Simulator hbm read test flow

hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus – receiveFromBus()

- sendToBank

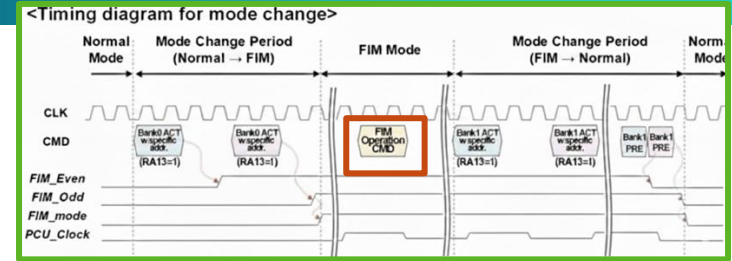
```

1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode == dramMode::HAB_PIM && pimRank->doPIM(packet));
9.             else
10.                pimRank->readHab(packet);
11.            packet->busPacketType = DATA;
12.            readReturnPacket.push_back(packet);
13.            readReturnCountdown.push_back(config.RL);
14.            break;
15.        case WRITE:
16.            if (mode == dramMode::SB)
17.                writeSb(packet);
18.            else if (mode == dramMode::HAB_PIM && pimRank->doPIM(packet));
19.            else
20.                pimRank->writeHab(packet);
21.            delete (packet);
22.            break;
23.        case ACTIVATE:
24.            if (DEBUG_CMD_TRACE)
25.            {
26.                PRINTC(getModeColor(), OUTLOG_ALL("ACTI"));
27.            }
28.            if (mode == dramMode::SB && packet->row == packet->column == 0x1f)
29.            {
30.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Odd_ = (packet->bank == 1) ? true : abmr2Even_ = (packet->bank == 8) ? true : abmr2Odd_ = (packet->bank == 9) ? true : false;
31.                if ((config.NUM_BANKS == 2 && abmr1Even_ && abmr1Odd_) || (config.NUM_BANKS == 2 && abmr2Even_ && abmr2Odd_))
32.                {
33.                    mode = dramMode::HAB;
34.                    if (DEBUG_CMD_TRACE)
35.                    {
36.                        PRINTC(RED, OUTLOG_CH_RA("HAB"));
37.                    }
38.                }
39.            }
40.            delete (packet);
41.            break;
42.    }
43. }

1. void PIMRank::writeHab(BusPacket* packet)
2. {
3.     if (packet->row == config.PIM_REG_RA) // WRIO to PIM Broadcasting
4.     {
5.         if (packet->column == 0x00)
6.             controlPIM(packet);
7.         if (0x08 <= packet->column && packet->column <= 0x0f || (0x18 <= packet->column && packet->column <= 0x1f))
8.         {
9.             if (DEBUG_CMD_TRACE)
10.            {
11.                if (packet->column - 8 < 8)
12.                    PRINTC(GREEN, OUTLOG_B_GRP_A("BWRITE_GRP_A"));
13.                else
14.                    PRINTC(GREEN, OUTLOG_B_GRP_B("BWRITE_GRP_B"));
15.            }
16.            #ifndef NO_STORAGE
17.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
18.            {
19.                if (packet->column - 8 < 8)
20.                    pimBlocks[pb].grfA[1] = packet->data->u8Data[20];
21.                else
22.                    pimBlocks[pb].grfB[1] = packet->data->u8Data[21];
23.            }
24.        }
25.        #endif
26.        else if (0x04 <= packet->column && packet->column <= 0x07 || (0x14 <= packet->column && packet->column <= 0x17))
27.        {
28.            if (DEBUG_CMD_TRACE)
29.            {
30.                PRINTC(GREEN, OUTLOG_B_GRP_A("BWRITE_GRP_A"));
31.            }
32.            #ifndef NO_STORAGE
33.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
34.            {
35.                if (packet->column - 4 < 4)
36.                    pimBlocks[pb].grfA[1] = packet->data->u8Data[20];
37.                else
38.                    pimBlocks[pb].grfB[1] = packet->data->u8Data[21];
39.            }
40.        }
41.        #endif
42.        else // PIM (only GRF)
43.        {
44.            PRINTC(GREEN, OUTLOG_B_GRP_A("BWRITE_GRP_A"));
45.            #ifndef NO_STORAGE
46.            int grf_id = getGrfId(packet->column);
47.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
48.            {
49.                if (packet->column - 4 < 4)
50.                    pimBlocks[pb].grfA[1] = packet->data->u8Data[20];
51.                else
52.                    pimBlocks[pb].grfB[1] = packet->data->u8Data[21];
53.            }
54.        }
55.        #endif
56.    }
57. }

1. void PIMRank::controlPIM(BusPacket* packet)
2. {
3.     uint8_t grf_a_zeroize = packet->data->u8Data[20];
4.     if (grf_a_zeroize)
5.     {
6.         if (DEBUG_CMD_TRACE)
7.         {
8.             PRINTC(RED, OUTLOG_CH_RA("GRF_A_ZEROIZE"));
9.         }
10.        BurstType burst_zero;
11.        for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
12.        {
13.            for (int i = 0; i < 8; i++) pimBlocks[pb].grfA[i] = burst_zero;
14.        }
15.    }
16.    uint8_t grf_b_zeroize = packet->data->u8Data[21];
17.    if (grf_b_zeroize)
18.    {
19.        if (DEBUG_CMD_TRACE)
20.        {
21.            PRINTC(RED, OUTLOG_CH_RA("GRF_B_ZEROIZE"));
22.        }
23.        BurstType burst_zero;
24.        for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
25.        {
26.            for (int i = 0; i < 8; i++) pimBlocks[pb].grfB[i] = burst_zero;
27.        }
28.    }
29.    pimOpMode_ = packet->data->u8Data[0] & 1;
30.    toggleEvenBank_ = !(packet->data->u8Data[16] & 1);
31.    toggleOddBank_ = !(packet->data->u8Data[16] & 2);
32.    toggleRa13h_ = (packet->data->u8Data[16] & 4);
33.    if (pimOpMode_)
34.    {
35.        rank->mode_ = dramMode::HAB_PIM;
36.        pimPC_ = 0;
37.        lastJumpIdx_ = numJumpToBeTaken_ = lastRepeatIdx_ = numRepeatToBeDone_ = -1;
38.        crfExit_ = false;
39.        PRINTC(RED, OUTLOG_CH_RA("HAB_PIM"));
40.    }
41.    else
42.    {
43.        rank->mode_ = dramMode::HAB;
44.        PRINTC(RED, OUTLOG_CH_RA("HAB mode"));
45.    }
46. }

```



command type별 처리 진행

2.3. Write → PIM mode 인지 판별 → 아닐 시 → writeHab() 수행

writeHab() → controlPIM()

- GRF 초기화

- u8Data[20]: GRF_A Reg를 0으로 초기화하는 flag
- u8Data[21]: GRF_B Reg를 0으로 초기화하는 flag

- PIM trigger bit 설정

- pimOpMode_ = packet->data->u8Data[0] & 1

- 1이면 PIM 연산 mode

- 0이면 HAB mode

- toggleEven/OddBank_ → isToggleCond()에서 사용
 - 각각 even/odd Bank에 대한 접근 시 doPIM()을 호출할지 여부를 결정

- toggleRa13h_ → isToggleCond()에서 사용

- doPIM() 호출 시 isReservedRA 여부를 고려할 지를 결정

- PIM 동작 mode 전환 (HAB ↔ HAB_PIM)

Samsung PIM Simulator hbm read test flow

hbm read test flow – measureCycle()

- measureCycle() → ... → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus – receiveFromBus()

- sendToBank

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode == dramMode::HAB_PIM && pimRank->doPIM(packet));
9.             else
10.                pimRank->readHab(packet);
11.            packet->busPacketType = DATA;
12.            readReturnPacket.push_back(packet);
13.            readReturnCountdown.push_back(config.RL);
14.            break;
15.        case WRITE:
16.            if (mode == dramMode::SB)
17.                writeSb(packet);
18.            else if (mode == dramMode::HAB_PIM && pimRank->doPIM(packet));
19.            else
20.                pimRank->writeHab(packet);
21.            delete (packet);
22.            break;
23.        case ACTIVATE:
24.            if (DEBUG_CMD_TRACE)
25.            {
26.                PRINTC(getModeColor(), OUTLOG_ALL("ACTI"));
27.            }
28.            if (mode == dramMode::SB && packet->row == packet->column == 0x1f)
29.            {
30.                abmr1Even_ = (packet->bank == 0) ? true : true;
31.                abmr1Odd_ = (packet->bank == 1) ? true : true;
32.                abmr2Even_ = (packet->bank == 8) ? true : true;
33.                abmr2Odd_ = (packet->bank == 9) ? true : true;
34.                if ((config.NUM_BANKS == 2 && abmr1Even_ && config.NUM_BANKS > 2 && abmr1Even_))
35.                {
36.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
37.                    mode = dramMode::HAB;
38.                    if (DEBUG_CMD_TRACE)
39.                    {
40.                        PRINTC(RED, OUTLOG_CH_RA("HAB"));
41.                    }
42.                }
43.                delete (packet);
44.                break;
45.            }
46.        }
47.    }
48.    delete (packet);
49.    break;
50. }
```

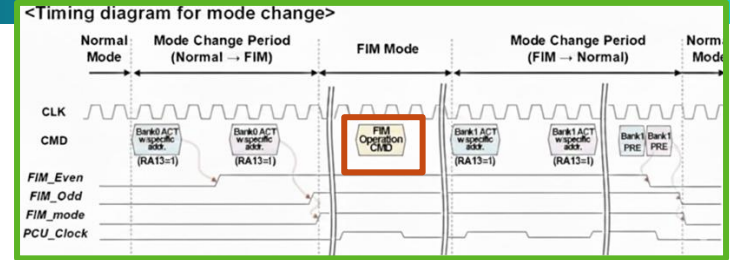
```
1. void PIMRank::writeHab(BusPacket* packet)
2. {
3.     if (packet->row == config.PIM_REG_RA) // WRIO to PIM Broadcasting
4.     {
5.         if (packet->column == 0x00)
6.             controlPIM(packet);
7.         if ((0x08 <= packet->column && packet->column <= 0x0f) ||
8.             (0x18 <= packet->column && packet->column <= 0x1f))
9.         {
10.            if (DEBUG_CMD_TRACE)
11.            {
12.                if (packet->column - 8 < 8)
13.                    PRINTC(GREEN, OUTLOG_B_GRF_A("BWRITE_GRF_A"));
14.                else
15.                    PRINTC(GREEN, OUTLOG_B_GRF_B("BWRITE_GRF_B"));
16.            }
17.            #ifndef NO_STORAGE
18.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
19.            {
20.                if (packet->column - 8 < 8)
21.                    pinBlocks[pb].grfA[packet->column - 0x8] = *(packet->data);
22.                else
23.                    pinBlocks[pb].grfB[packet->column - 0x18] = *(packet->data);
24.            }
25.            #endif
26.        }
27.        else if (0x04 <= packet->column && packet->column <= 0x07)
28.        {
29.            if (DEBUG_CMD_TRACE)
30.            {
31.                PRINTC(GREEN, OUTLOG_B_CRF("BWRITE_CRF"));
32.            }
33.            crf.bst[packet->column - 0x04] = *(packet->data);
34.        }
35.        else if (packet->column == 0x1)
36.        {
37.            if (DEBUG_CMD_TRACE)
38.            {
39.                PRINTC(GREEN, OUTLOG_CH_RA("BWRITE_SRF"));
40.            }
41.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++) pinBlocks[pb].srf = *(packet->data);
42.        }
43.        else if (isReservedRA(packet->row))
44.        {
45.            PRINTC(GRAY, OUTLOG_ALL("WRITE"));
46.        }
47.        else // PIM (only GRF) to Bank Move
48.        {
49.            PRINTC(GREEN, OUTLOG_ALL("PIM_TO_BANK"));
50.        }
51.        #ifndef NO_STORAGE
52.        int grf_id = getGrfIdx(packet->column);
53.        for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
54.        {
55.            if (packet->bank == 0)
56.            {
57.                *(packet->data) = pinBlocks[pb].grfA[grf_id];
58.                rank->banks[pb * 2].write(packet); // basically read from bank;
59.            }
60.            else if (packet->bank == 1)
61.            {
62.                *(packet->data) = pinBlocks[pb].grfB[grf_id];
63.                rank->banks[pb * 2 + 1].write(packet); // basically read from bank.
64.            }
65.        }
66.    }
67. }
```

Reference: Hardware_Architecture_ar

PIMRank.cpp

case WRITE

Rank.cpp



command type별 처리 진행

2.3. Write → PIM mode 인지 판별 → 아닐 시 → writeHab() 수행

writeHab()

- Row Address가 PIM_REG_RA와 동일 시

- column == 0x00: controlPIM() 함수를 호출
- column 0x08 ~ 0x0f || 0x18 ~ 0x1f
 - 모든 PIM Block의 GRF에 동일 data write
- column 0x04 ~ 0x07
 - CRF(command register file)에 data write
- column 0x1
 - 모든 PIM 블록의 SRF(scalar register file)에 동일 data write

- isReservedRA()==true 시

- Write를 skip

- 그 외

- GRF Register에서 even/odd DRAM Bank로 저장

```
PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

Configuration.h

```
BurstType srf;
BurstType grfA[8];
BurstType grfB[8];
```

PIMBlock.h

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

abcd

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case ACTIVATE

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.            {
65.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.
68.                if (sbmr1_ && sbmr2_)
69.                {
70.                    sbmr1_ = sbmr2_ = false;
71.                    mode_ = dramMode::SB;
72.                    if (DEBUG_CMD_TRACE)
73.                    {
74.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
75.                    }
76.                }
77.            }
78.            delete (packet);
79.            break;
80.        case REF:
81.            refreshWaiting = false;
82.            if (DEBUG_CMD_TRACE)
83.            {
84.                PRINT(OUTLOG_CH_RA("REF"));
85.            }
86.            delete (packet);
87.            break;
88.        case DATA:
89.            delete (packet);
90.            break;
91.        default:
92.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
93.            exit(0);
94.            break;
95.    }
96. }
97.
98. Rank.cpp
99.
100.
101. }
```


Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

abcd

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case PRECHARGE

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.            {
65.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.
68.                if (sbmr1_ && sbmr2_)
69.                {
70.                    sbmr1_ = sbmr2_ = false;
71.                    mode_ = dramMode::SB;
72.                    if (DEBUG_CMD_TRACE)
73.                    {
74.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
75.                    }
76.                }
77.            }
78.            delete (packet);
79.            break;
80.        case REF:
81.            refreshWaiting = false;
82.            if (DEBUG_CMD_TRACE)
83.            {
84.                PRINT(OUTLOG_CH_RA("REF"));
85.            }
86.            delete (packet);
87.            break;
88.        case DATA:
89.            delete (packet);
90.            break;
91.        default:
92.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
93.            exit(0);
94.            break;
95.    }
96. }
97.
98. Rank.cpp
99.
100.
101. }
```


Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

abcd

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case REF

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.            {
65.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.
68.                if (sbmr1_ && sbmr2_)
69.                {
70.                    sbmr1_ = sbmr2_ = false;
71.                    mode_ = dramMode::SB;
72.                    if (DEBUG_CMD_TRACE)
73.                    {
74.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
75.                    }
76.                }
77.            }
78.            delete (packet);
79.            break;
80.        case REF:
81.            refreshWaiting = false;
82.            if (DEBUG_CMD_TRACE)
83.            {
84.                PRINTC(OUTLOG_CH_RA("REF"));
85.            }
86.            delete (packet);
87.            break;
88.        case DATA:
89.            delete (packet);
90.            break;
91.        default:
92.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
93.            exit(0);
94.            break;
95.    }
96. }
97.
98. Rank.cpp
99.
100.
101. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

abcd

- sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case DATA

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
64.            {
65.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
66.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
67.
68.                if (sbmr1_ && sbmr2_)
69.                {
70.                    sbmr1_ = sbmr2_ = false;
71.                    mode_ = dramMode::SB;
72.                    if (DEBUG_CMD_TRACE)
73.                    {
74.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
75.                    }
76.                }
77.            }
78.            delete (packet);
79.            break;
80.        case REF:
81.            refreshWaiting = false;
82.            if (DEBUG_CMD_TRACE)
83.            {
84.                PRINT(OUTLOG_CH_RA("REF"));
85.            }
86.            delete (packet);
87.            break;
88.        case DATA:
89.            delete (packet);
90.            break;
91.        default:
92.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
93.            exit(0);
94.            break;
95.    }
96. }
97.
98. Rank.cpp
99.
100.
101. }
```

Samsung PIM Simulator hbm read test flow

□ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()

3. Write Data Bus

- Host(MC)에서 DRAM(Rank)으로 가는 Write Data 흐름을 관리
- dataCyclesLeft: data Burst가 유지되는 시간을 추적 → WriteDataDone: counter가 0이 되면, MC는 bus transfer가 끝났음을 MemorySystem에 알림
- Transaction 관리: numOnTheFlyTransactions를 줄여서, 현재 처리 중인 작업 하나가 끝났음을 기록

```
1. void MemoryController::update()
2. {
3.     updateBankState();
4.     // check for outgoing command packets and handle countdowns
5.     if (outgoingCmdPacket != NULL)
6.     {
7.         cmdCyclesLeft--;
8.         if (cmdCyclesLeft == 0) // packet is ready to be received by rank
9.         {
10.            (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
11.            outgoingCmdPacket = NULL;
12.        }
13.    }
14.
15.    // check for outgoing data packets and handle countdowns
16.    if (outgoingDataPacket != NULL)
17.    {
18.        dataCyclesLeft--;
19.        if (dataCyclesLeft == 0)
20.        {
21.            // inform upper levels that a write is done
22.            if (parentMemorySystem->WriteDataDone != NULL)
23.            {
24.                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
25.                outgoingDataPacket->physicalAddress,
26.                currentClockCycle);
27.            }
28.            parentMemorySystem->numOnTheFlyTransactions--;
29.            (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
30.            outgoingDataPacket = NULL;
31.        }
32.    }
33. }
```

[cf] packet 구성

```
cmdPacket
패킷 구성:
  busPacketType: ACTIVATE
  physicalAddress: 0x12345600
  rank: 0
  bank: 2
  row: 1024
  column: (사용 안 함)
  data: NULL

WRITE 를 위한 DataPacket
패킷 구성:
  busPacketType: DATA
  physicalAddress: 0x12345600
  rank: 0
  bank: 2
  row: 1024
  column: 8
  data: [0xAF, 0xBE, 0xAD, ...] (64 바이트 분량의 실제 값) or NULL
```

Samsung PIM Simulator gemv test flow

□ gemv test flow

- Samsung PIM Simulator는 GoogleTest(gtest)기반의 simulation test를 수행함
- 따라서, 이 flow를 Samsung PIM Simulator가 gemv operation을 어떻게 test하는지 따라가며 알아본다

./sim --gtest_filter=PIMKernelFixture.gemv

```
1. #include <bitset>
2. #include <cstdlib>
3. #include <iostream>
4. #include <limits>
5. #include <memory>
6. #include <random>
7.
8. #include "Burst.h"
9. #include "MultiChannelMemorySystem.h"
10. #include "gtest/gtest.h"
11. #include "tests/PIMKernel.h"
12. #include "tests/TestCases.h"
13.
14. using namespace std;
15.
16. int main(int argc, char* argv[])
17. {
18.     if (argc > 1)
19.     {
20.         ::testing::InitGoogleTest(&argc, argv);
21.     }
22.     else
23.     {
24.         ::testing::InitGoogleTest(&argc, argv);
25.         testing::GTEST_FLAG(filter) = "-*bw*.*";
26.     }
27.
28.     return RUN_ALL_TESTS();
29. }
```

MainTest.cpp

```
1. TEST_F(PIMKernelFixture, gemv)
2. {
3.     shared_ptr<PIMKernel> kernel = make_pim_kernel();
4.
5.     uint32_t batch_size = 1;
6.     uint32_t output_dim = 4096;
7.     uint32_t input_dim = 1024;
8.
9.     DataDim *dim_data = new DataDim(KernelType::GEMV, batch_size, output_dim, input_dim, true);
10.    dim_data->printDim(KernelType::GEMV);
11.
12.    reduced_result_ = new BurstType[dim_data->dimTobShape(output_dim)];
13.    result_ = getResultPIM(KernelType::GEMV, dim_data, kernel, result_);
14.
15.    testStatsClear();
16.    expectAccuracy(KernelType::GEMV, output_dim, dim_data->output_npbst_,
17.                  dim_data->getNumElementsPerBlocks());
18.
19.    delete[] result_;
20.    delete[] reduced_result_;
21.    delete dim_data;
22. }
```

KernelTestCases.cpp

Samsung PIM Simulator gemv test flow

□ gemv test flow

- Samsung PIM Simulator는 GoogleTest(gtest)기반의 simulation test를 수행함
- 따라서, 이 flow를 Samsung PIM Simulator가 gemv operation을 어떻게 test하는지 따라가며 알아본다

```
1. TEST_F(PIMKernelFixture, gemv)
2. {
3.     shared_ptr<PIMKernel> kernel = make_pim_kernel();
4.
5.     uint32_t batch_size = 1;
6.     uint32_t output_dim = 4096;
7.     uint32_t input_dim = 1024;
8.
9.     DataDim *dim_data = new DataDim(KernelType::GEMV, batch_size, output_dim, input_dim, true);
10.    dim_data->printDim(KernelType::GEMV);
11.
12.    reduced_result_ = new BurstType[dim_data->dimTobShape(output_dim)];
13.    result_ = getResultPIM(KernelType::GEMV, dim_data, kernel, result_);
14.
15.    testStatsClear();
16.    expectAccuracy(KernelType::GEMV, output_dim, dim_data->output_npbst_,
17.                  dim_data->getNumElementsPerBlocks());
18.
19.    delete[] result_;
20.    delete[] reduced_result_;
21.    delete dim_data;
22. }
```

KernelTestCases.cpp

```
1.     shared_ptr<PIMKernel> make_pim_kernel()
2.     {
3.         shared_ptr<MultiChannelMemorySystem> mem = make_shared<MultiChannelMemorySystem>(
4.             "ini/HBM2_samsung_2M_16B_x64.ini", "system_hbm_64ch.ini", ".", "example_app",
5.             256 * 64 * 2);
6.         int numPIMChan = 64;
7.         int numPIMRank = 1;
8.         shared_ptr<PIMKernel> kernel = make_shared<PIMKernel>(mem, numPIMChan, numPIMRank);
9.
10.        return kernel;
11.    }
```